GERMAN INTERNATIONAL UNIVERSITY

Micro-Controllers:

AVR Architecture + In/Out

Dr. Eng. Amr T. Abdel-Hamid

Spring 2022

Processor technology

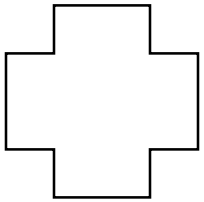
- Processors vary in their customization for the problem at hand

Desired functionality

```
total = 0
for i = 1 to N  loop
   total += M[i]
end loop
```

General-purpose processor

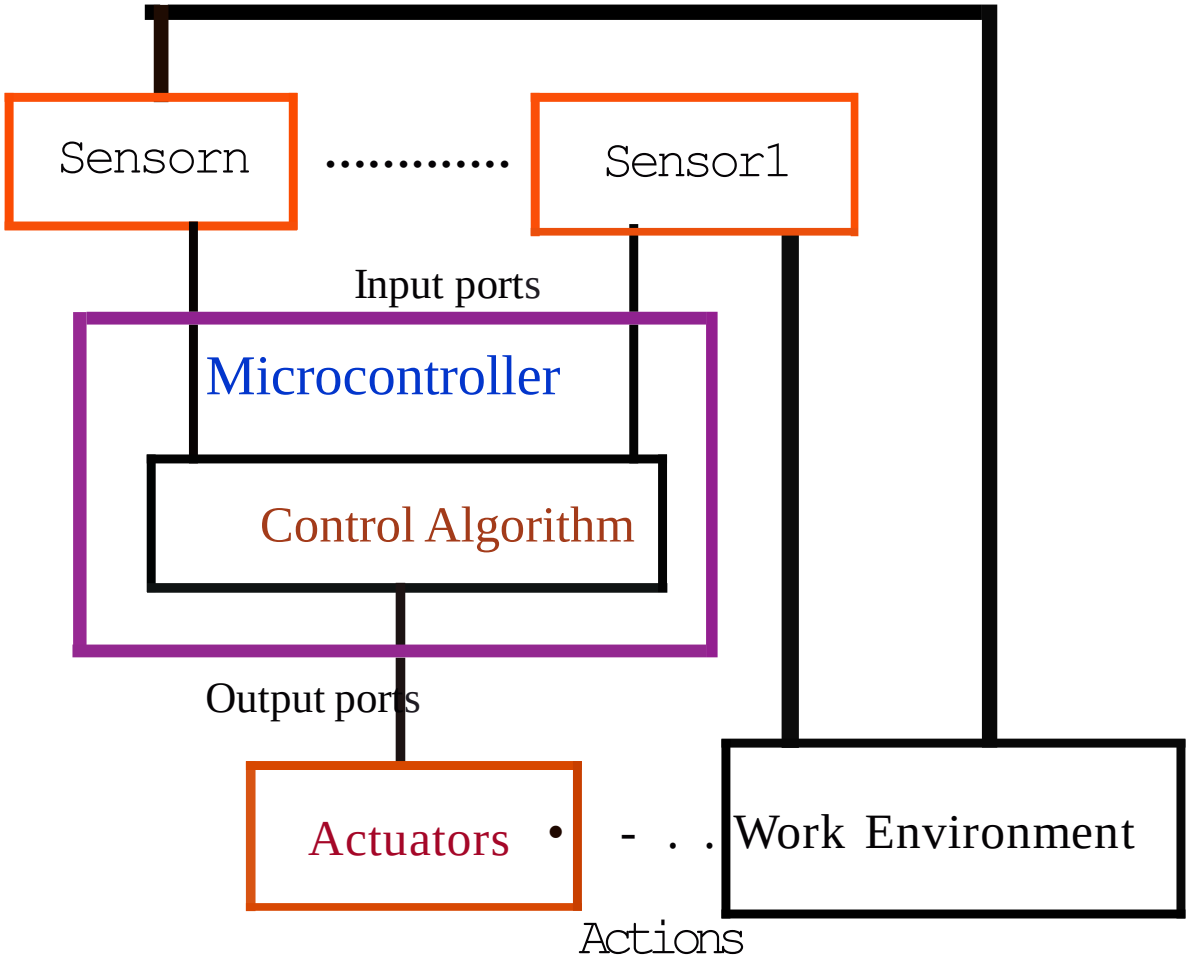
Application-specific processor



Single-purpose processor

Dr. Amr Talaat

Architecture of an Embedded System



Sense and Action loop through Microcontroller

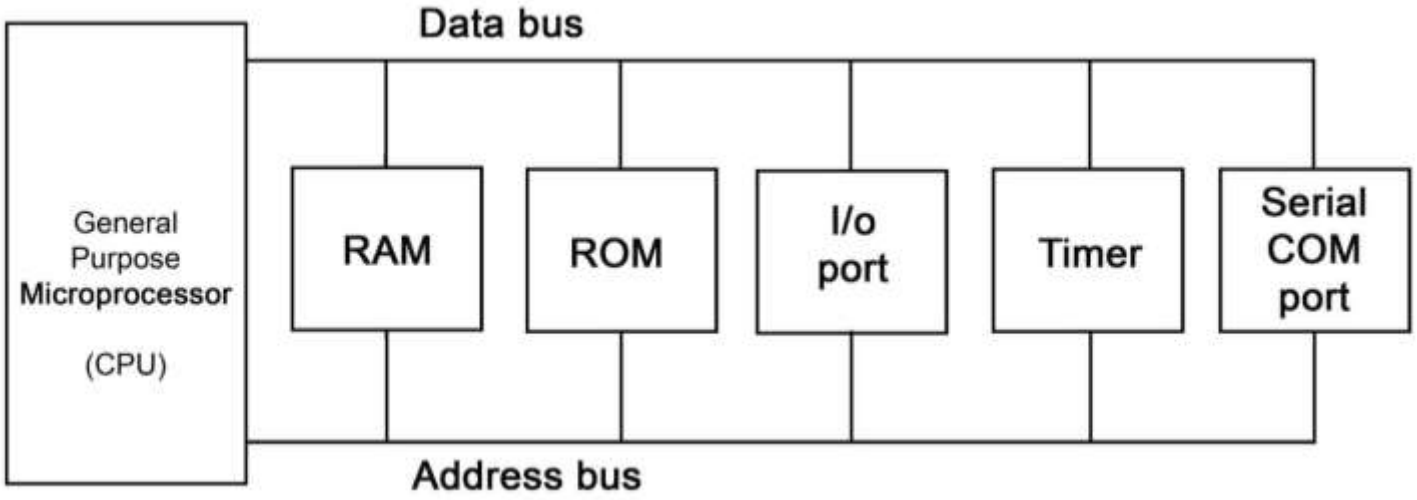
Dr. Amr Talaat

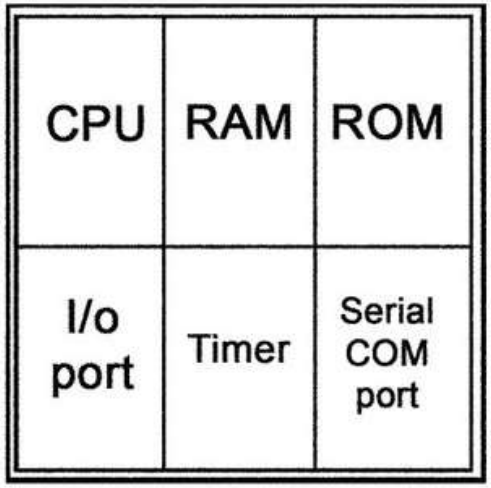
Microprocessors and Microcontrollers

- A microprocessor unit (MPU) is a processor on one silicon chip.
- Microcontrollers are used in embedded computing.
- A Microcontroller unit (MCU) is a microprocessor with some added circuitry on one silicon chip.

Dr. Amr Talaat

A Single Chip Microcontroller Contains





(Single chip)

Dr. Amr Talaat

MCU Types

- MCUs can be classified according:
  - **MCU bits** (e.g. 8-bit, 16-bit, 32-bit)
  - **Instruction Set Architecture** (ISA)

Dr. Amr Talaat

Why study the ARM architecture (and the Cortex-M3 in particular)?

Lots of manufacturers ship ARM products

Actel® POWER MATTERS

ATMEL®

CYPRESS PERFORM

DUST NETWORKS

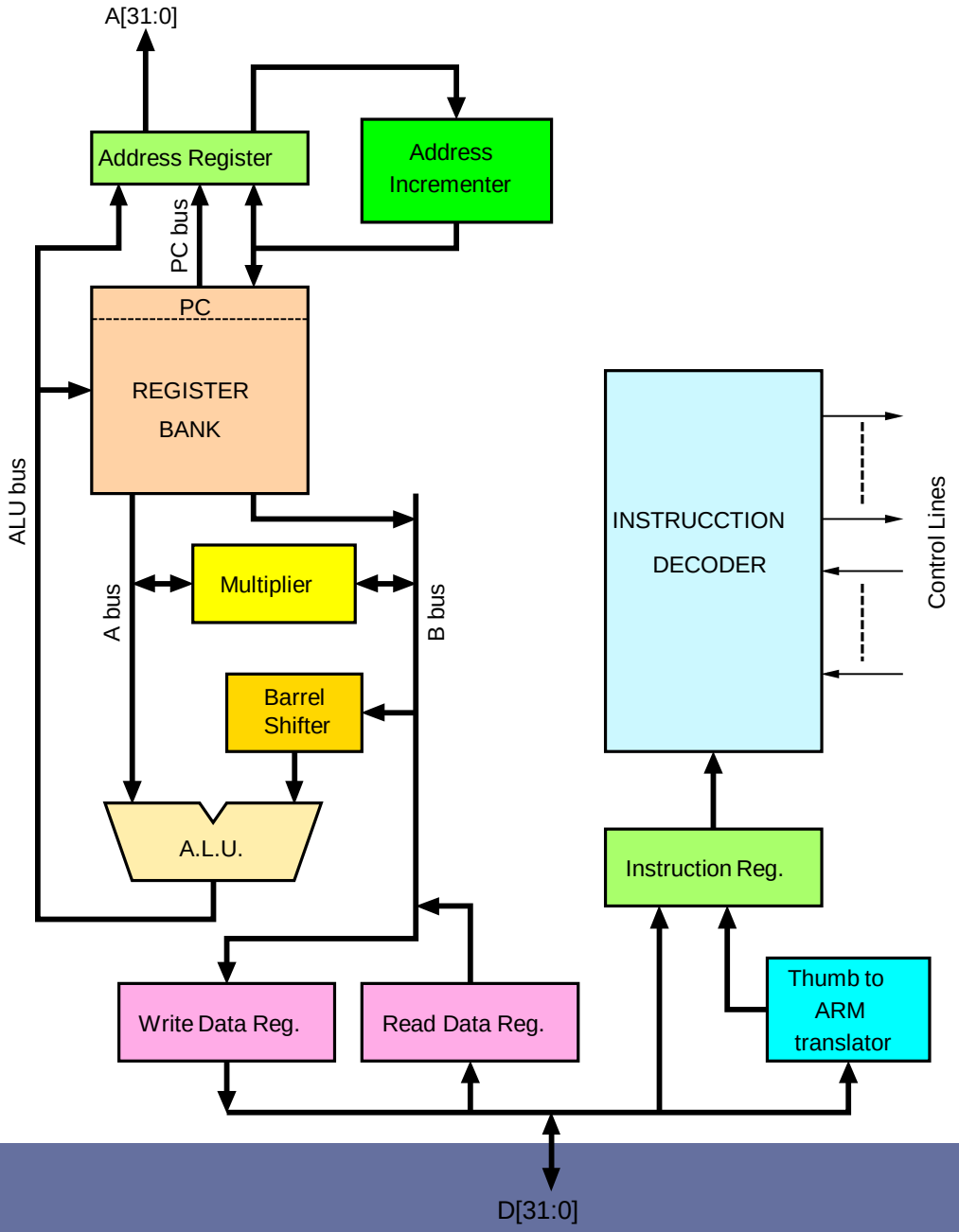
ember

NXP

TEXAS INSTRUMENTS

Dr. Amr Talaat

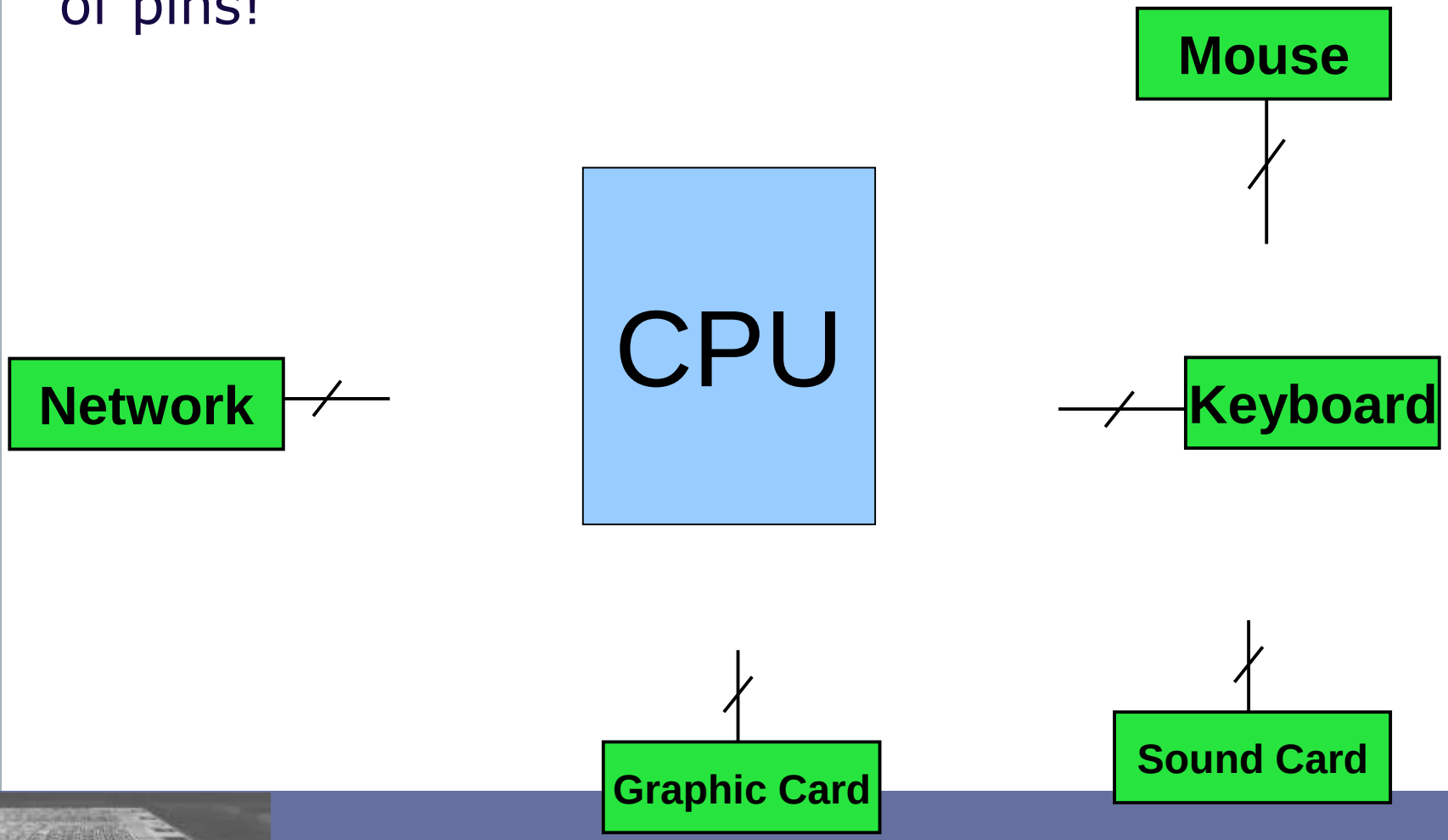
ARM7TDMI Block Diagram



Dr. Amr Talaat

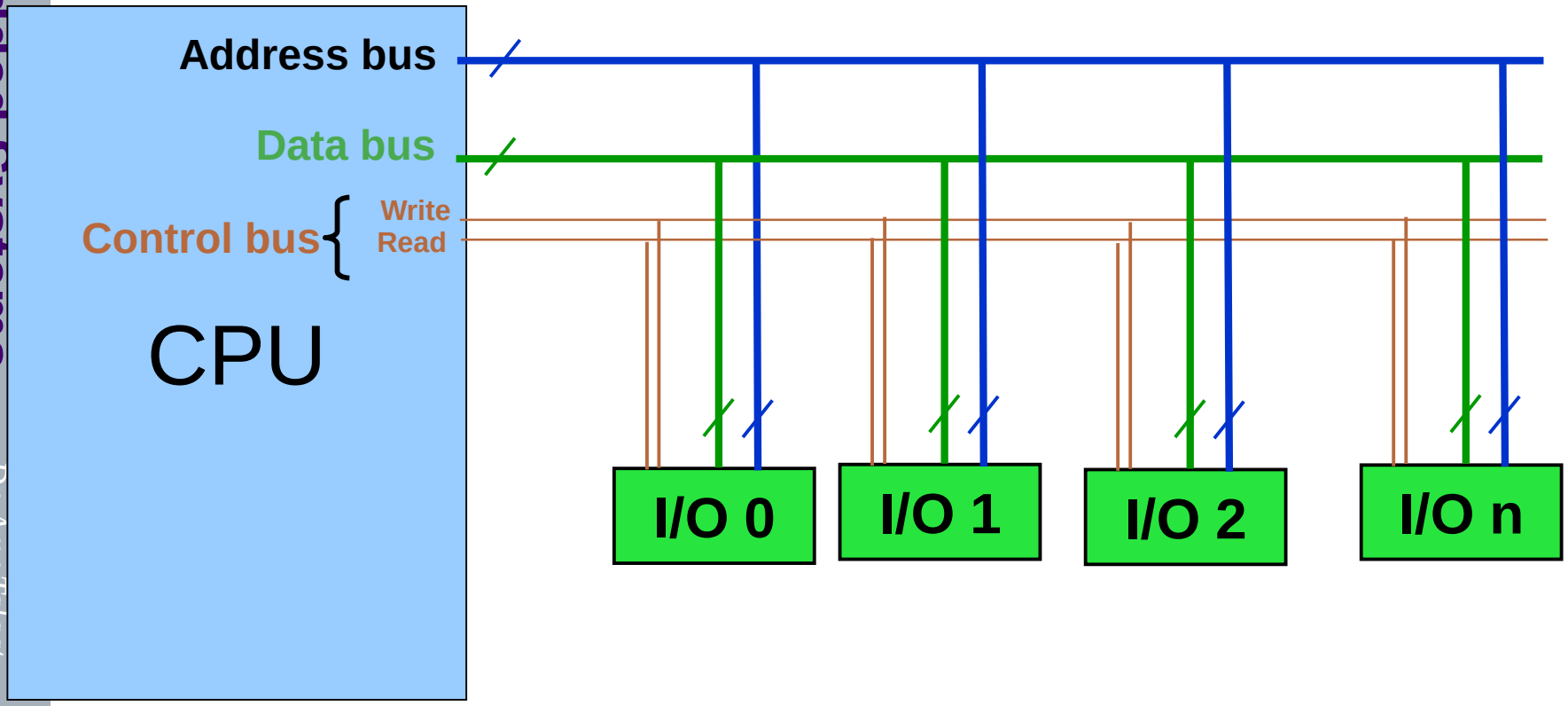
Connecting I/Os to CPU

- CPU should have lots of pins!



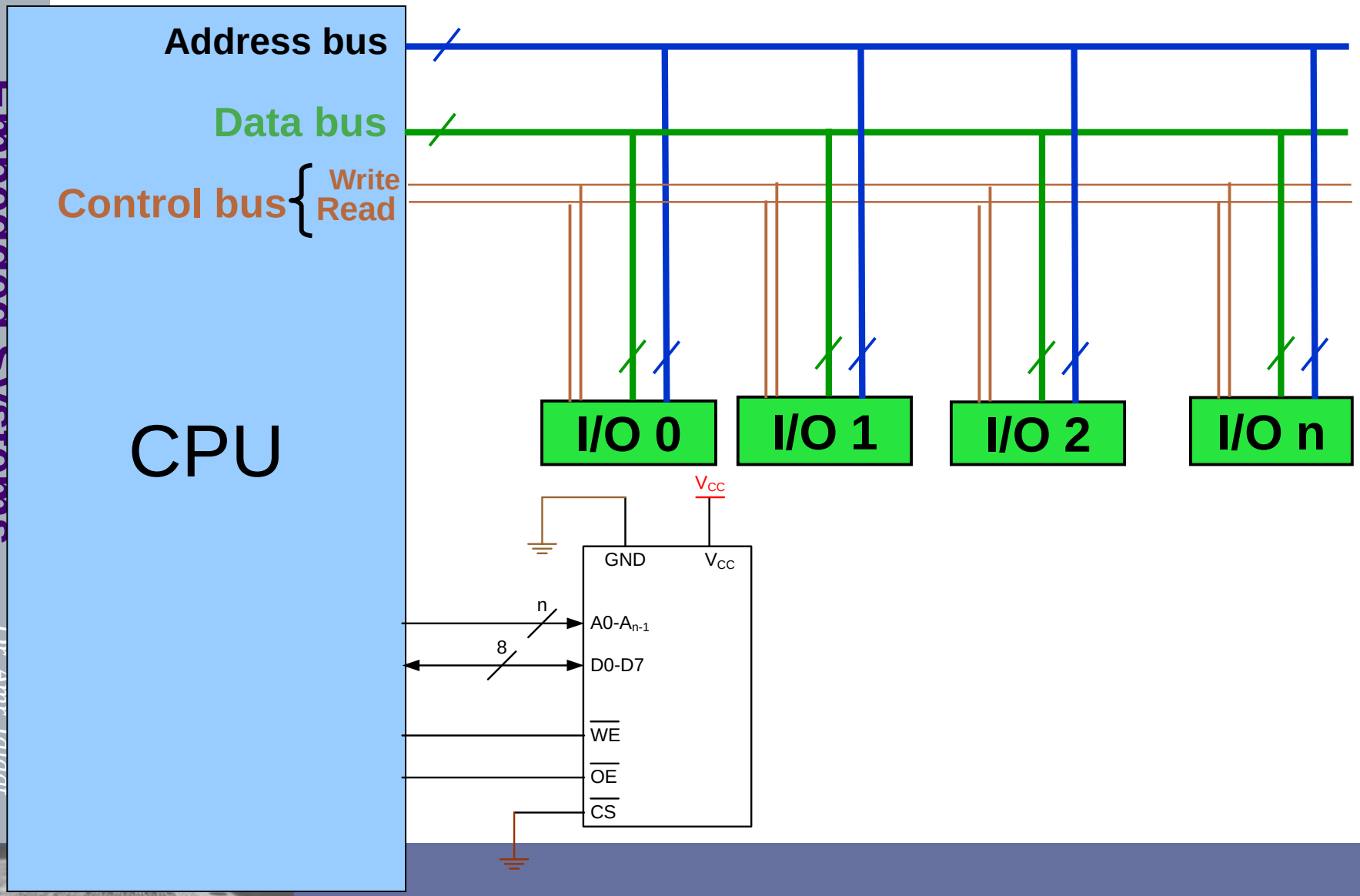
Dr. Amr Talaat

Connecting I/Os to CPU using bus



Dr. Amr Talaat

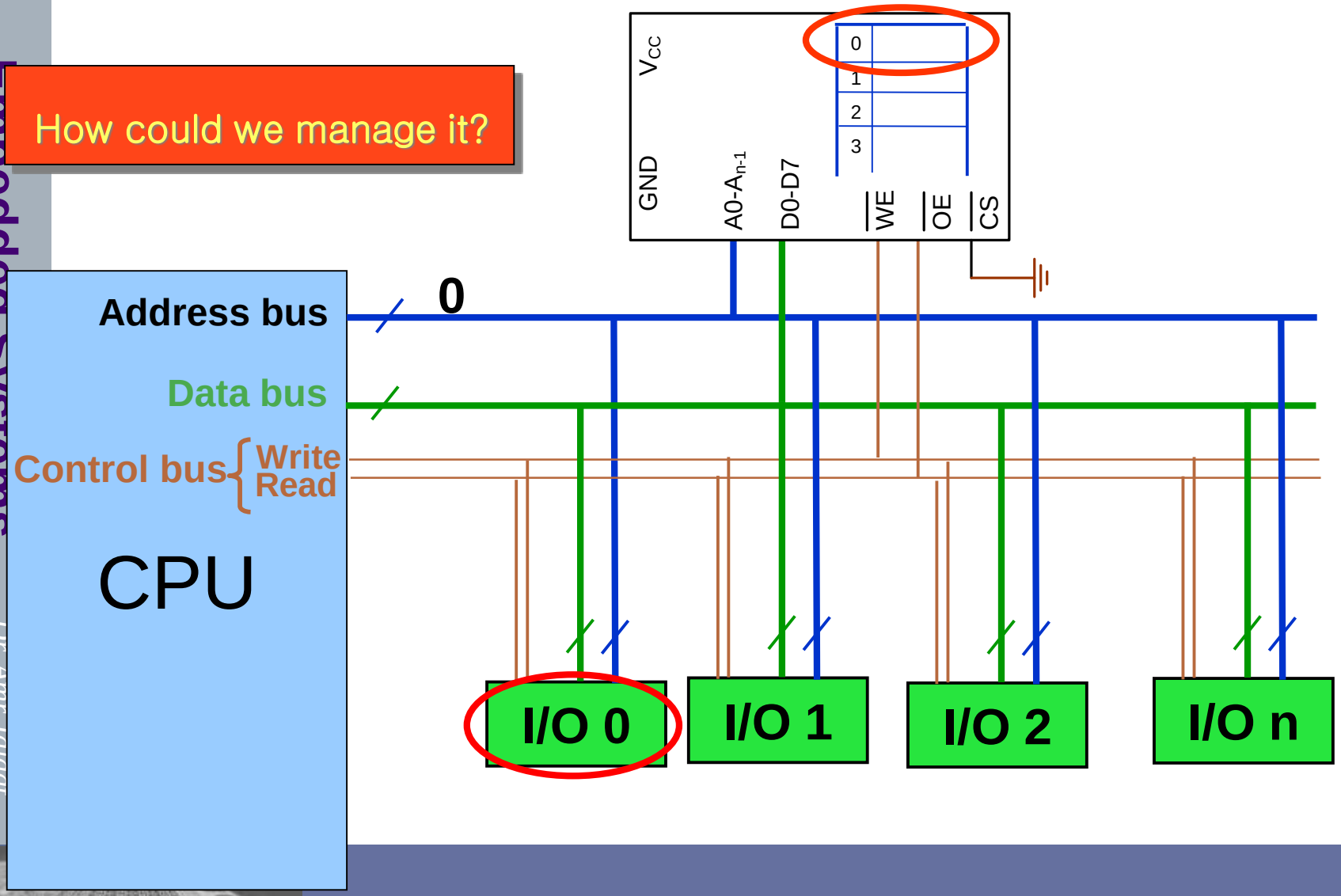
Connecting I/Os and Memory to CPU



Dr. Amr Talaat

Connecting I/Os and memory to CPU using bus

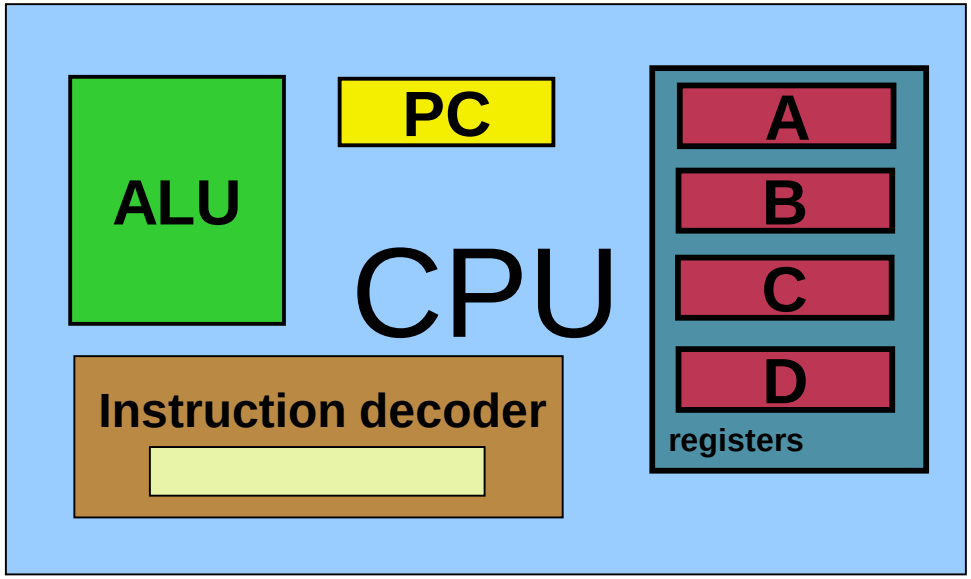
How could we manage it?



Dr. Amr Talaat

Inside the CPU

- PC (Program Counter)
- Instruction decoder
- ALU (Arithmetic Logic Unit)
- Registers



Dr. Amr Talaat

What will a Program do?

- Add, Subtract,…
- And, Or, Compare….
- For, While,…
- IF, then else

Dr. Amr Talaat

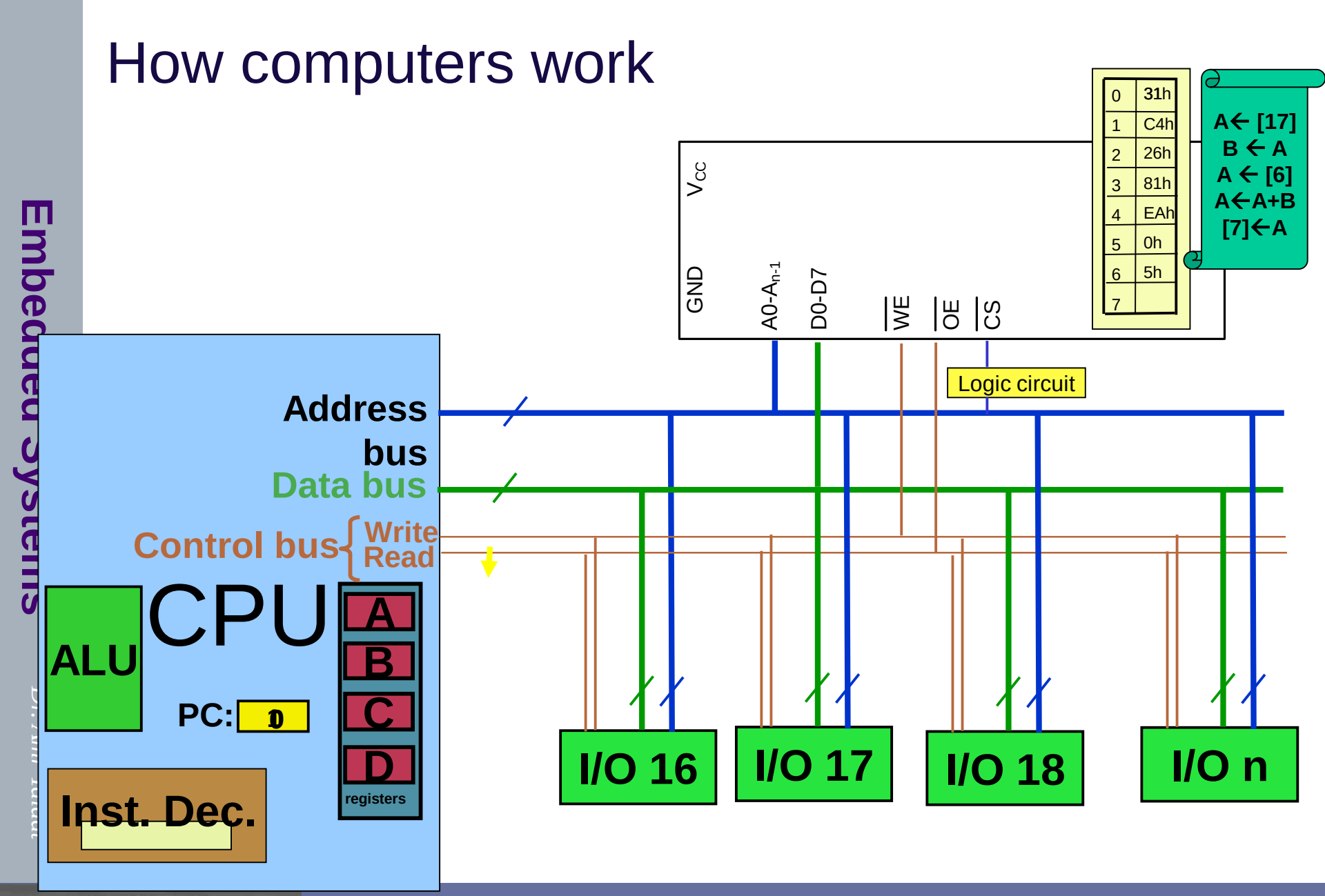
Programming Languages Main Functions

- Main Language Functions:
  - An arithmetic operation
  - A logic operation
  - Storing a datum
  - Moving a datum
  - Jump or branch (to implement the IF and For)
- How to implement these above functions?
  - Adder, Subtractor, Multiplier, …
  - And, Or, Xor, Not, ….

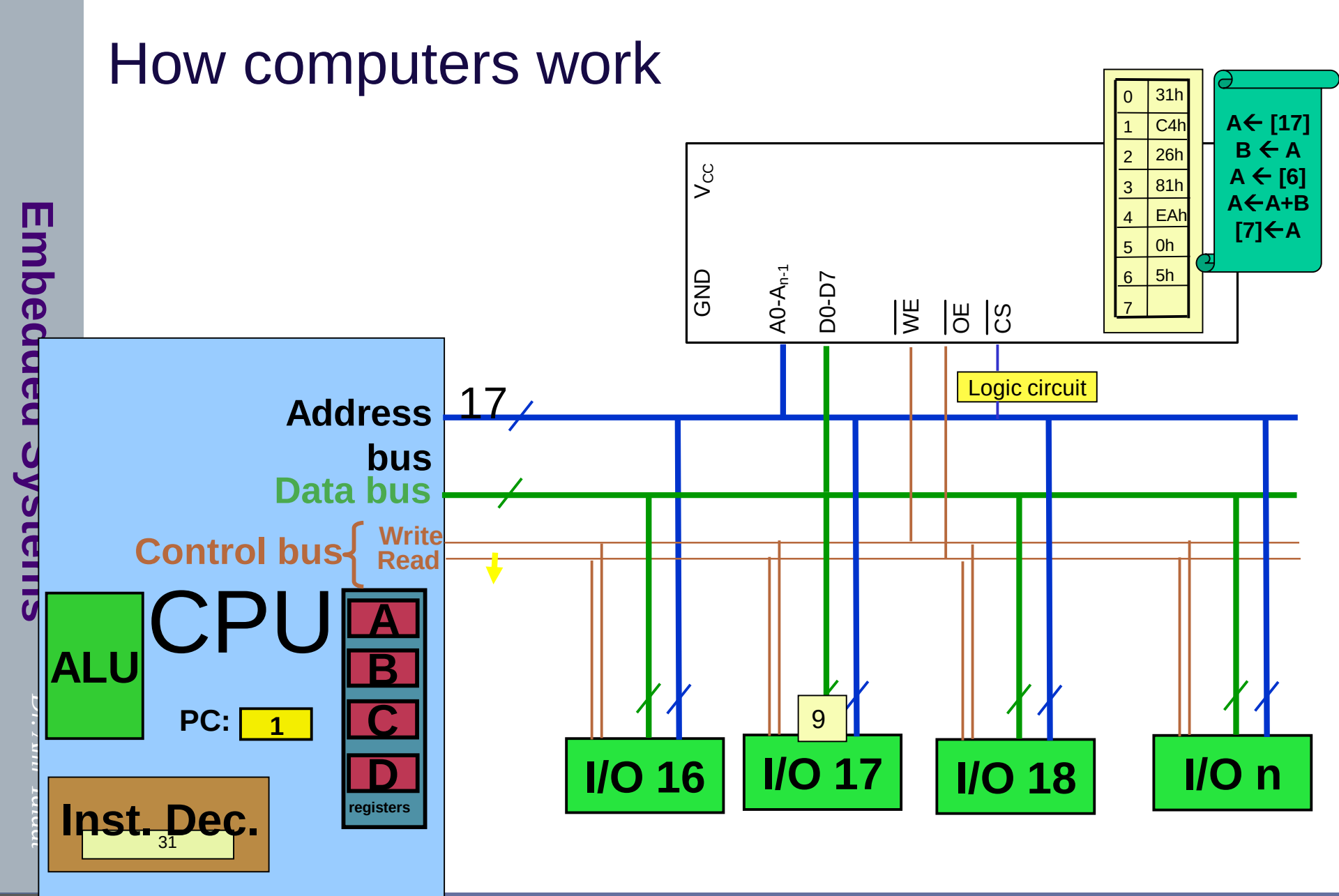
The Arithmetic Logic Unit, AKA **ALU**

Dr. Amr Talaat

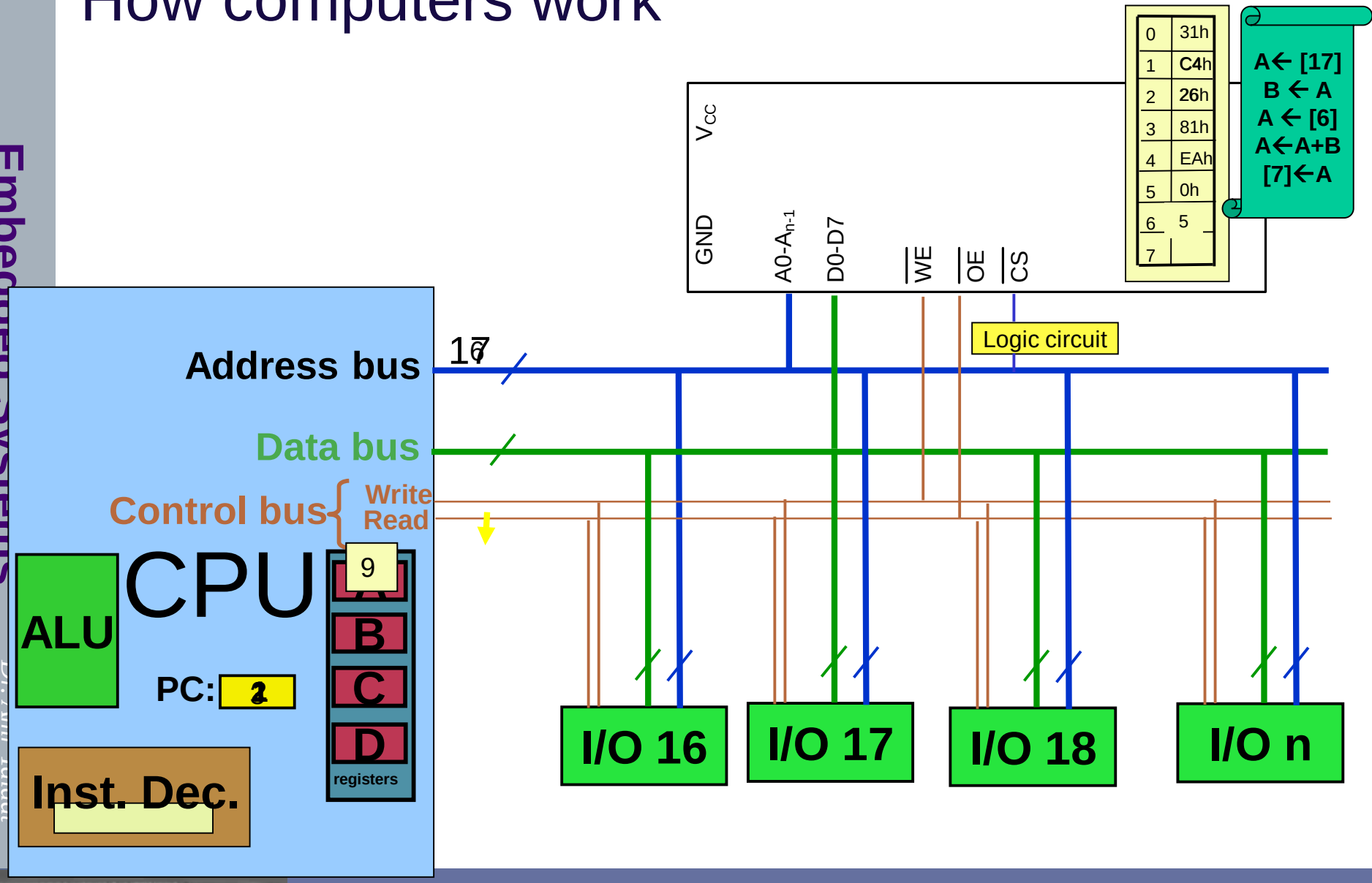
How computers work



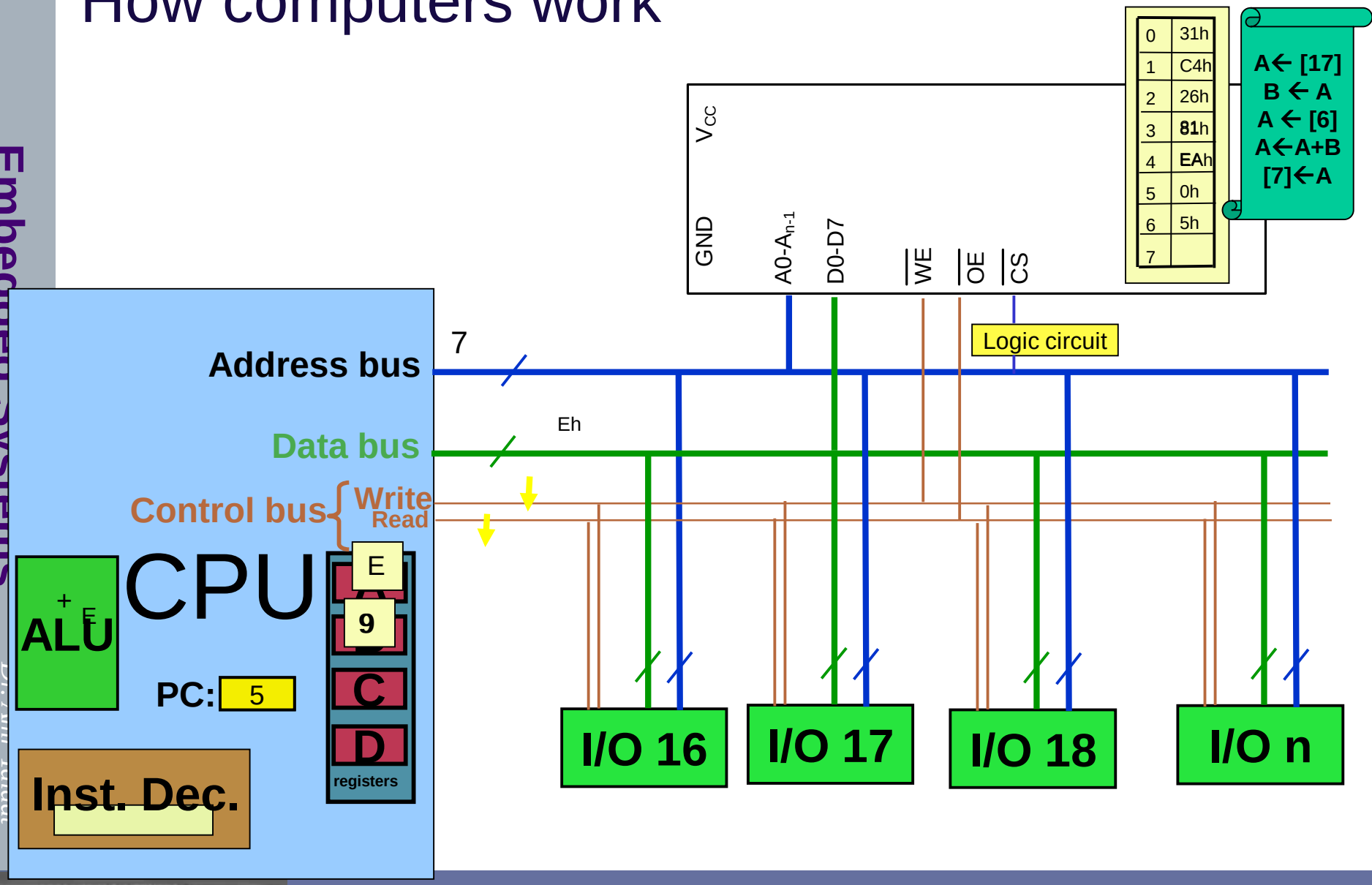
How computers work



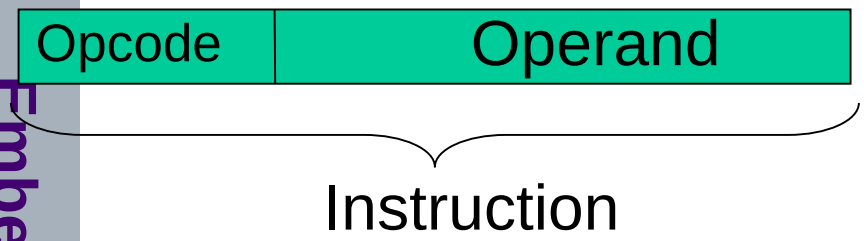
How computers work

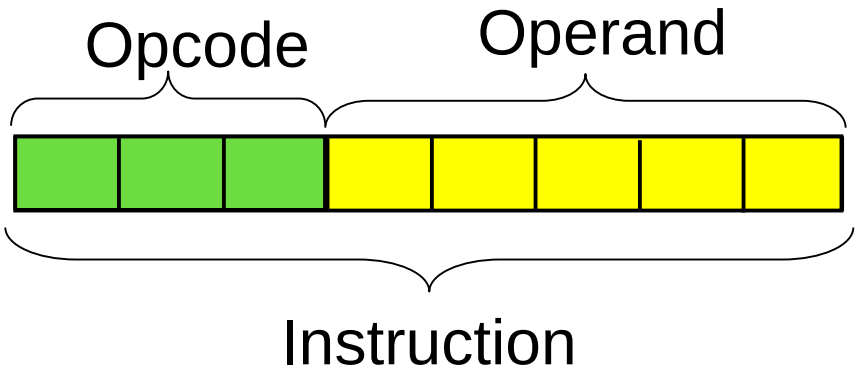


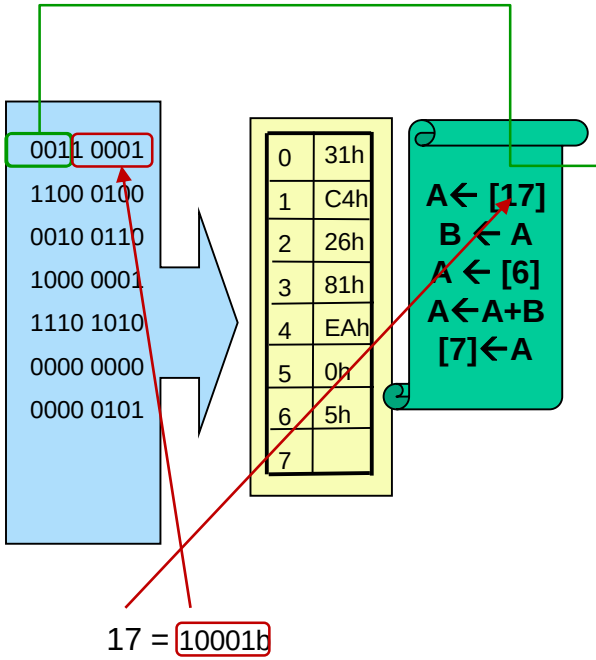
How computers work



How Instruction decoder works



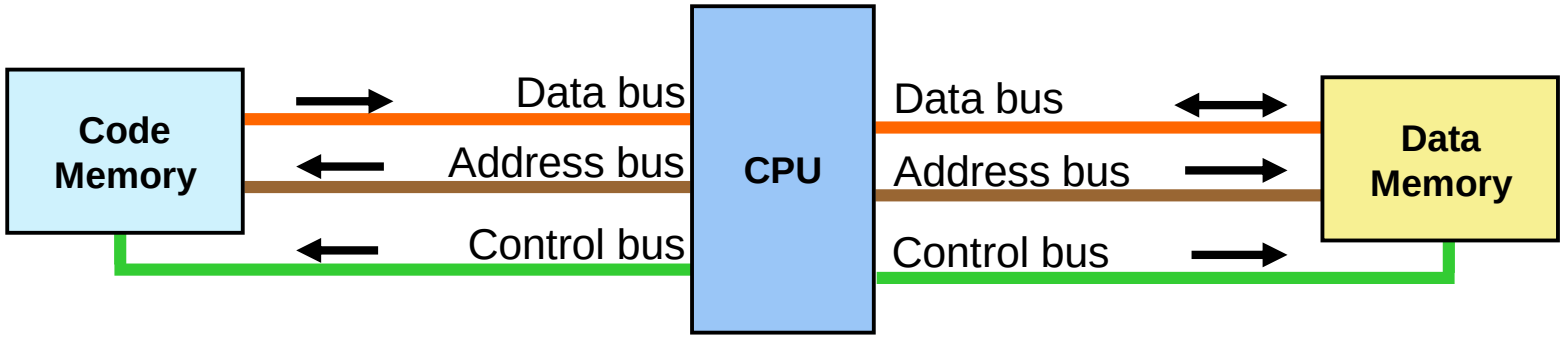




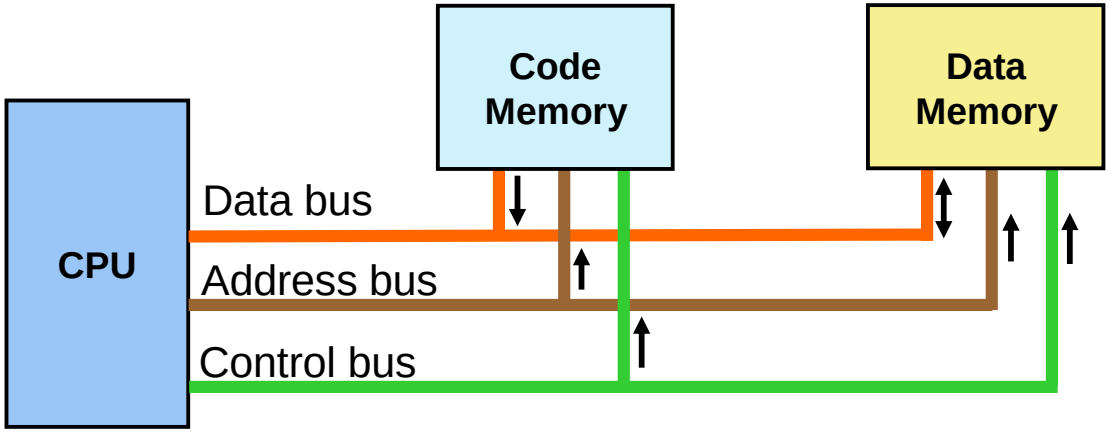
Operation Code	Meaning
000	A ← x
001	A ← [x]
010	A ← A – register (x)
011	A ← A + x
100	A ← A + register (x)
101	A ← A – x
110	Register (x_H) ← Register (x_L)
111	[x] ← A

Dr. Amr Talaat

Von Neumann vs. Harvard architecture



- Von Neumann architecture

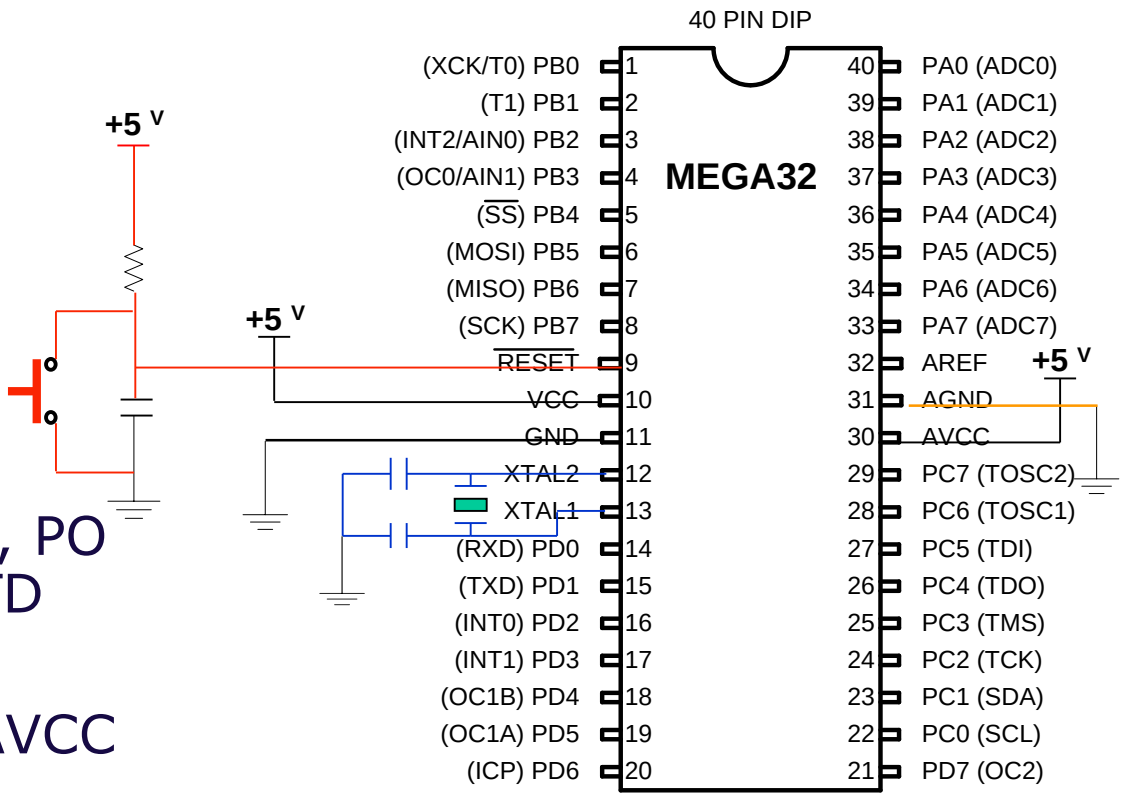


- Harvard architecture

Dr. Amr Talaat

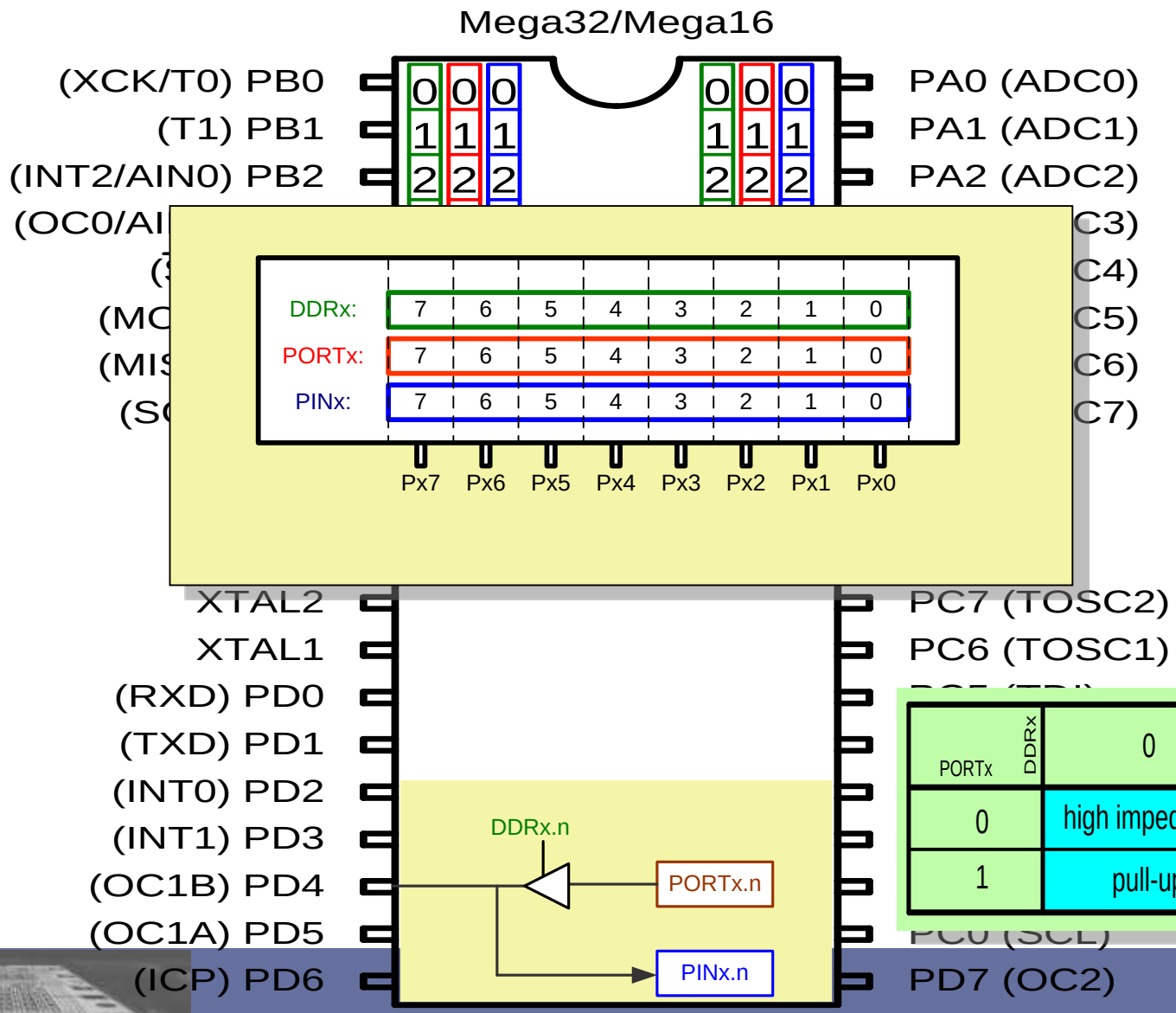
ATmega16/mega32 pinout

1. Vital Pins:
   1. Power
      - VCC
      - Ground
   2. Crystal
      - XTAL1
      - XTAL2
   3. Reset
2. I/O pins
   - PORTA, PORTB, PORTC, and PORTD
3. Internal ADC pins
   - AREF, AGND, AVCC



Dr. Amr Talaat

The structure of IO pins



PORTx \ DDRx	0	1
0	high impedance	Out 0
1	pull-up	Out 1

Dr. Amr Talaat

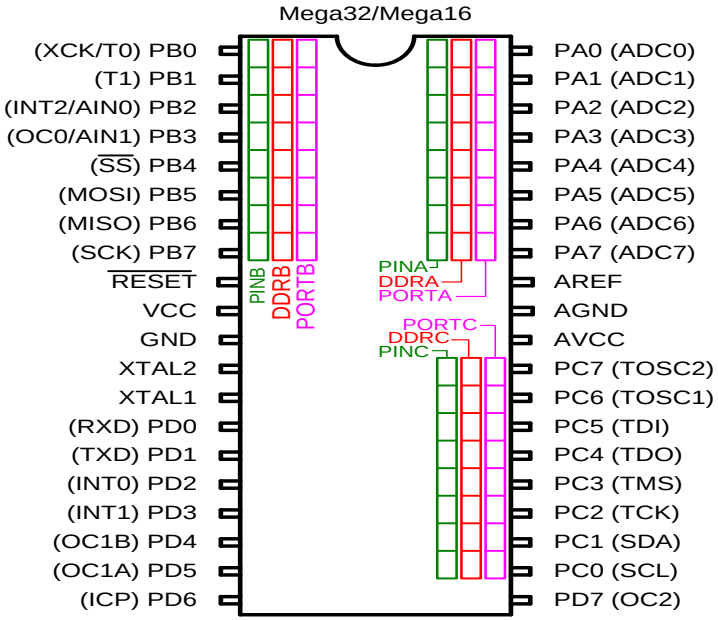
Example 1

- Write a program that makes all the pins of PORT A one.

DDRA:	1	1	1	1	1	1	1	1
PORTA:	1	1	1	1	1	1	1	1

```
.INCLUDE "M32DEF.INC"
LDI   R20,0xFF   ;R20 = 11111111 (binary)
OUT   PORTA,R20  ;PORTA = R20
OUT   DDRA,R20   ;DDRA = R20
```

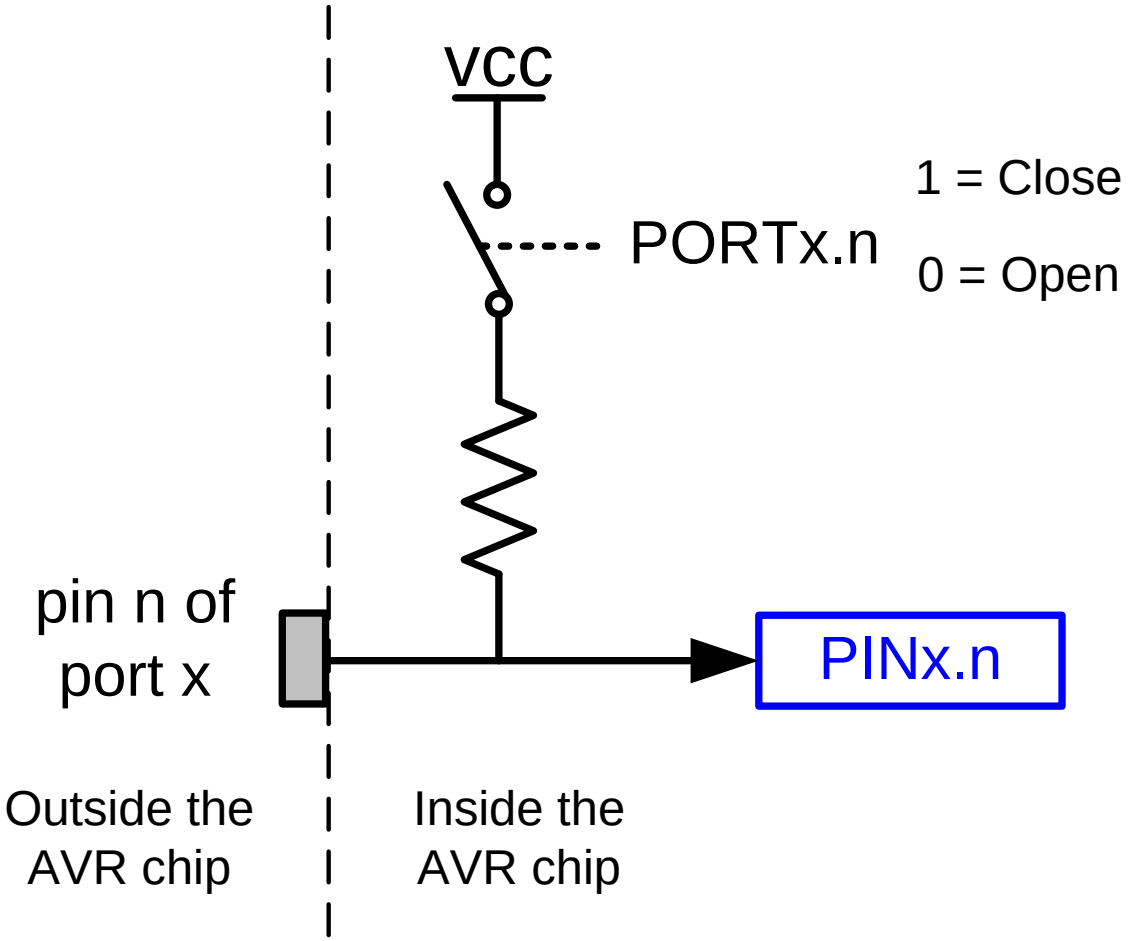
Register	Description
DDRx	Used to configure the respective PORT as output/input
PORTx	Used to write the data to the Port pins
PINx	Used to Read the data from the port pins



PORTx \ DDRx	0	1
0	high impedance	Out 0
1	pull-up	Out 1

Dr. Amr Talaat

Pull-up resistor



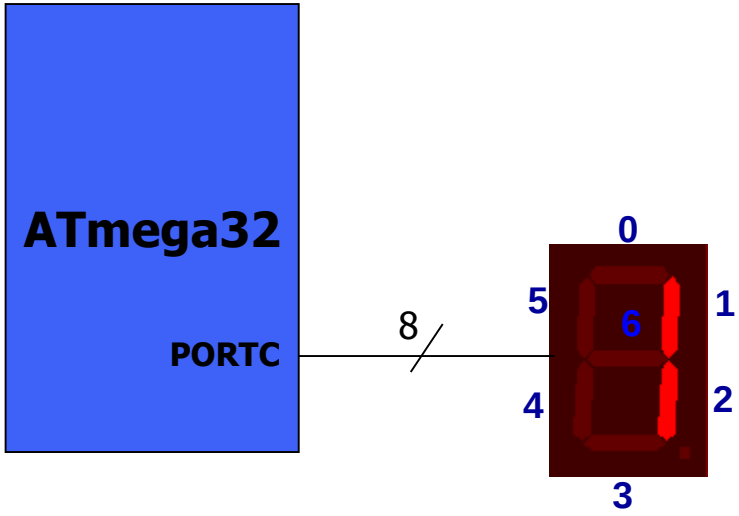
Dr. Amr Talaat

Example 3

- A 7-segment is connected to PORTA. Display 1 o n the 7-segment.

DDRC	1	1	1	1	1	1	1	1
PORTC:	0	0	0	0	0	1	1	0

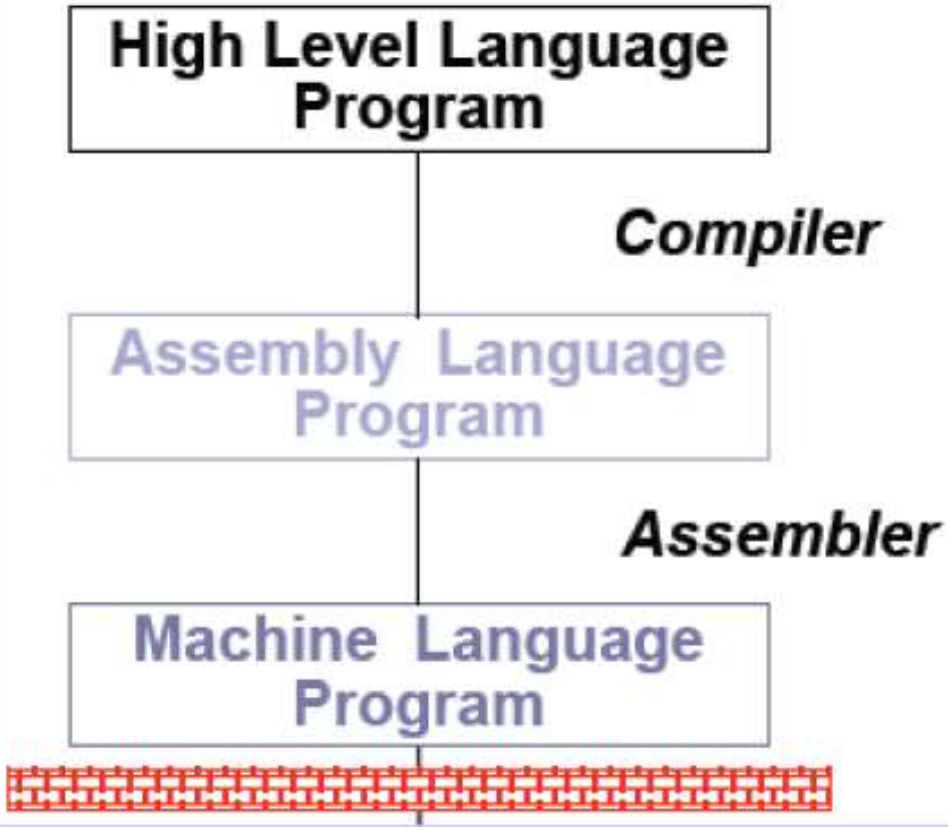
```
.INCLUDE "M32DEF.INC"
LDI   R20,0x06   ;R20 = 00000110 (binary)
OUT   PORTC,R20  ;PORTC = R20
LDI   R20,0xFF   ;R20 = 11111111 (binary)
OUT   DDRC,R20   ;DDRC = R20
L1: RJMP L1
```



PORTx \ DDRx	0	1
0	high impedance	Out 0
1	pull-up	Out 1

Dr. Amr Talaat

High level Language to Machine code



temp = v[k];

v[k] = v[k+1];

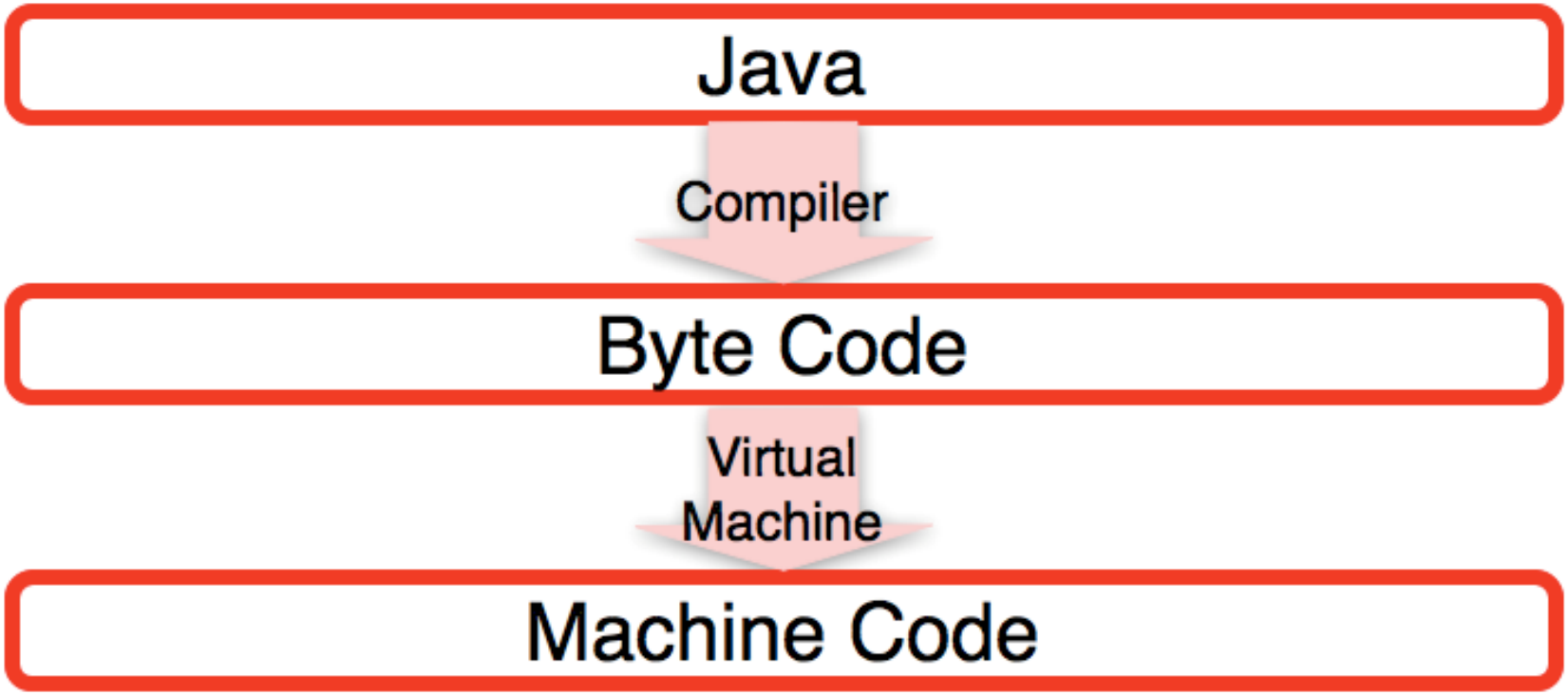
v[k+1] = temp;

- mov BL, 02h
- mov AX,0A56h
- mul BL

```
0000 1001 1100 0110 1010
1010 1111 0101 1000 0000
1100 0110 1010 1111 0101
0101 1000 0000 1001 1100
```


Dr. Amr Talaat

Java Language to Machine code



Dr. Amr Talaat

GAS program format

```
            .file "test.s"
            .text
            .global main
            .type main, %function
main:
            MOV R0, #100
            ADD R0, R0, R0
            SWI #11
            .end
```


Dr. Amr Talaat

The structure of the C code

```
/* any needed header files are included */
#include <avr/io.h>
#include <util/delay.h>

/* any definitions for constant values are written next */
#define RS 6
#define E 5

/* any needed function definitions go here */

void send_a_command(unsigned char command);
void send_a_character(unsigned char character);


int main(void)
{

    /* your main code goes here */

}

void send_a_command(unsigned char command)
{
    /* your code to send a command to the LCD goes here */
}

void send_a_character(unsigned char character)
{
    /* your code to send a text to the LCD goes here */
}
```

1. `#include` tells the compiler where to look for other bits of code you are using that are not included in the current file.

They are normally".h" or header files that contain pre-written macrosand function prototypes that are commonly used regardlessof the code functionality.

Before the .c file is compiled, the contents of the `#include` areliterallycopied into the file.

Dr. Amr Talaat

The structure of the C code

```c
/* any needed header files are included */
#include <avr/io.h>
#include <util/delay.h>

/* any definitions for constant values are written next */
#define RS 6
#define E 5

/* any needed function definitions go here */

void send_a_command(unsigned char command);
void send_a_character(unsigned char character);


int main(void)
{

    /* your main code goes here */

}

void send_a_command(unsigned char command)
{
    /* your code to send a command to the LCD goes here */
}

void send_a_character(unsigned char character)
{
    /* your code to send a text to the LCD goes here */
}
```

2. `#define` are "macros".

Before the programis compiled, the argument of the `#define` is simply substituted everywhere the name is used.

They are useful when:
•There arebits of code that areheavily repeated throughout the file
•Defining a constant value for a parameter used heavily throughout thecode.

This way we can easily change them fromthe defines section directly

Dr. Amr Talaat

Dr. Amr Talaat

```c
/* any needed header files are included */
#include <avr/io.h>
#include <util/delay.h>

/* any definitions for constant values are written next */
#define RS 6
#define E 5

/* any needed function definitions go here */

void send_a_command(unsigned char command);
void send_a_character(unsigned char character);

int main(void)
{

    /* your main code goes here */

}

void send_a_command(unsigned char command)
{
    /* your code to send a command to the LCD goes here */
}

void send_a_character(unsigned char character)
{
    /* your code to send a text to the LCD goes here */
}
```

The structure of the C code

3. The main function, as its name implies, is where the main code is executed.

The compiler will not accept a code without a main function.

Inside the main function, the code is executed sequentially line by line.

The main function is where we define the usage of the I/O ports of the micro- controller and initialize its different modules.

```c
/* any needed header files are included */
#include <avr/io.h>
#include <util/delay.h>

/* any definitions for constant values are written next */
#define RS 6
#define E 5

/* any needed function definitions go here */

void send_a_command(unsigned char command);
void send_a_character(unsigned char character);

int main(void)
{

    /* your main code goes here */

}

void send_a_command(unsigned char command)
{
    /* your code to send a command to the LCD goes here */
}

void send_a_character(unsigned char character)
{
    /* your code to send a text to the LCD goes here */
}
```

The structure of the C code

4. Inside the main function, any number of auxiliary functions can be used, to simplify the code and make it more compact

They can be void functions or they can return outputs of any type accordingto their use in the code

Dr. Amr Talaat

Include and definitions

DO YOU REMEMBER THIS LINE

```
import java.io.*;
```

In the previous java courses we used this line to import some predefined classes in java to be used in the active class we are working through

But, How we could do the same in C- Language?

```
#include <h_file> ;
```

It could be done using the #include statement to include one of the header files (h_files) that have all the microcontrollers predefined functions and register names

Dr. Amr Talaat

ALSO YOU CAN FIND IN C- LANGUAGE

#define X Y ;

It is used to define variable called X to have the same value or behavior as variable Y and this can be realized in

EXAMPLE

```
#include <AVR4534.h>  ;
/* this makes the header file for the your
ready to be used in the C-class */
#define LED PORTC ;
/* defines LED variable in the class to
work as PORTC so what happens to LED will
be applied on PORTC */
```

Class structure and main

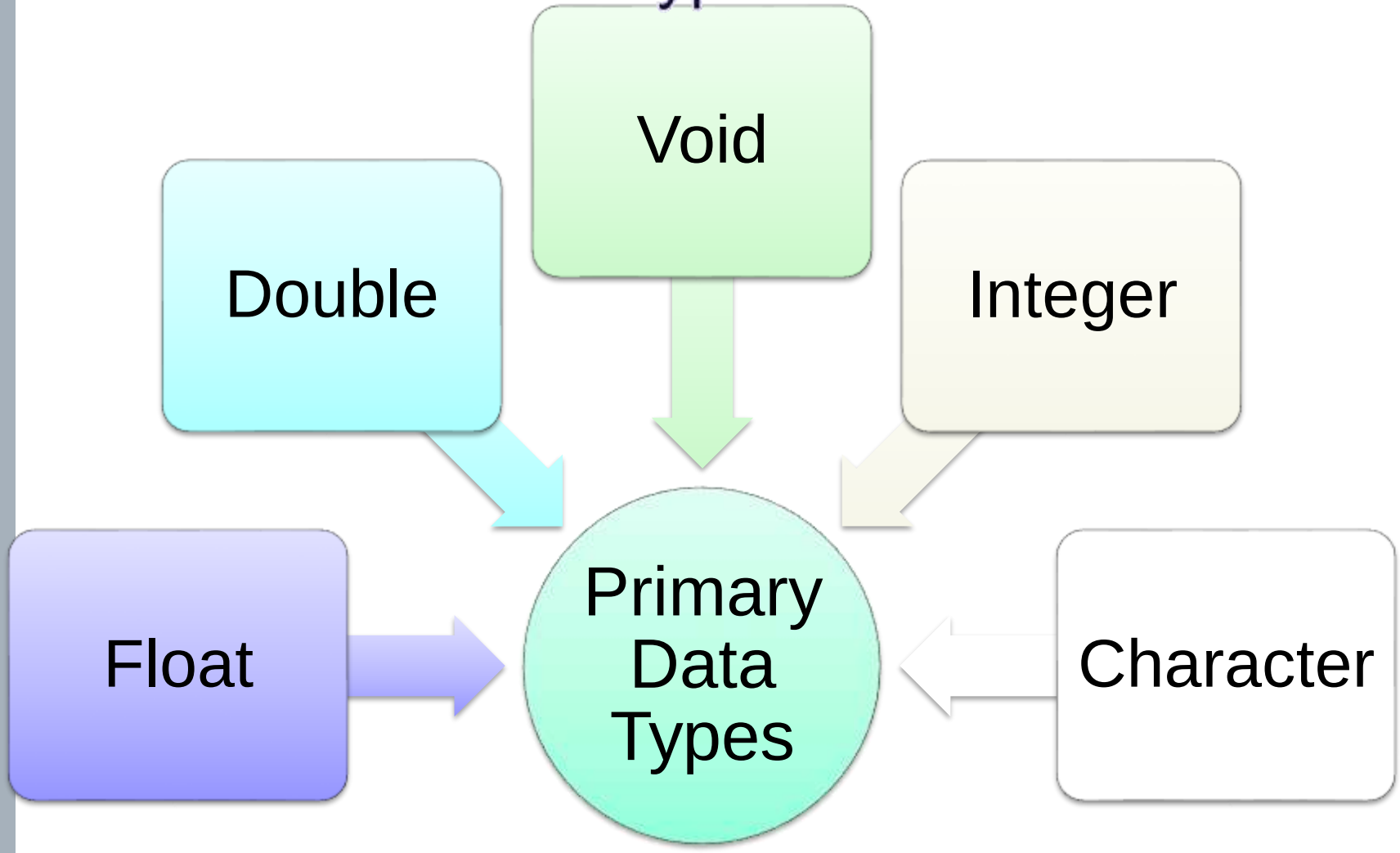
THE C CLASS CONSIST OF THE FOLLOWING

```
#include <pic16f877a.h>  ; // header file include
#define LED PORTC ;        // define LED as PORTC

void main ( void )         // main method
{
  // main method implementation and print test
   printf("hellow World");
}
```

- **MAIN METHOD is the one responsible for organizing the class methods and variables. It can call any other method inside**
- **VOID means it won't return any value because it is the main one (optional to write between brackets)**

Dr. Amr Talaat

Variables and Data types



Dr. Amr Talaat

VARIABLES AND DATA TYPES

Variable Type	Keyword	Bytes Required	Range
Character	char	1	-128 to 127
Unsigned character	unsigned char	1	0 to 255
Integer	int	2	-32768 to 32767
Short Integer	short int	2	-32768 to 32767
Long Integer	long int	4	-2,147,483,648 to 2,147,438,647
Unsigned Integer	unsigned int	2	0 to 65535
Unsigned Short integer	unsigned short int	2	0 to 65535
Unsigned Long Integer	unsigned long int	4	0 to 4,294,967,295
Float	float	4	1.2E-38 to
Double	double	8	2.2E-308 to
Long Double	long double	10	3.4E-4932 to 1.1E+4932

Variables and Data types

➔**Integer**

int is used to define integer numbers

Int count = 5;

➔**float**

FLOAT is used to define floating point numbers

float count = 5.6;

➔**char**

char is used to define characters

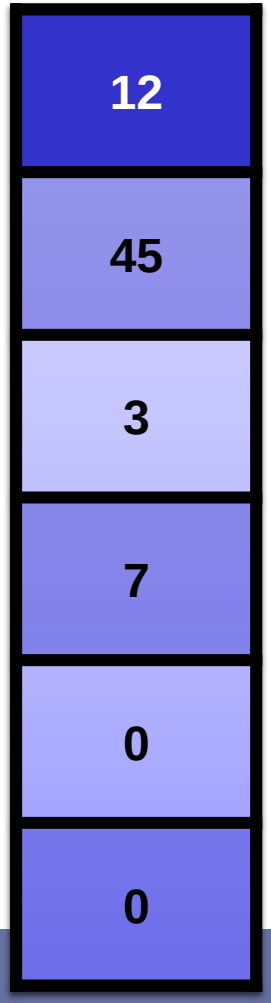
char letter = ‘x’;

Dr. Amr Talaat

Arrays

- Arrays is the group of elements saved in one group and each element has an index of its location that contains its value
- All of its elements should have the same type
- Index always starts from zero

```
int [6] x ;
/* Creates empty array of length 6 cells */
int [6] x = {12,45,3,7} ;
/* Creates array of length 6 cells shown in this page
where the cells values are the values between the  the
curly braces */
/* notice that the rest of cells that not declared yet
will have a value of 0 (default value of the integer
type) */
```



cont.

- **Two Dimensional Arrays**

```
int [6][2] x ;
/* Create empty 2D-array of 6 cells */
 x[1][1] = 12 ;
/* assign value 12 to cell (1,1)*/
 x[3][0] = 20 ;
/* assign value 12 to cell (3,0)*/
 x[5][1] = 10 ;
/* assign value 12 to cell (5,1)*/
 Int temp = x[1][1] ;
/* temp in memory will have value of 12*/
```

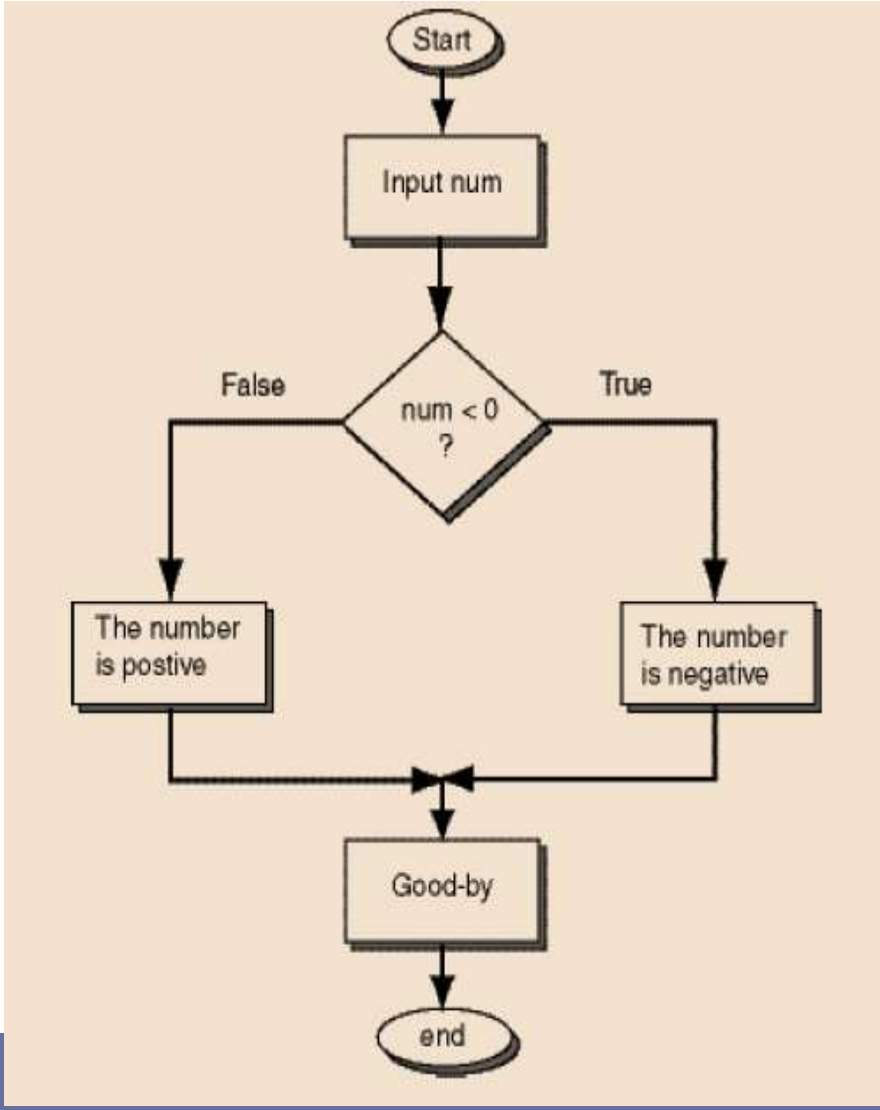
0	0
0	12
0	0
20	0
0	0
0	10

Dr. Amr Talaat

Conditional Statement

Conditional statement gives you an option to make branches in the evaluated condition .. If the condition is true it would take certain otherwise it would take the other one like the following flow chart.

```
int num = 6 ;

if ( num < 0)
{
   printf("negative");
}
else
{
   printf("positive");
}
```



cont.

Conditional statement has two forms

switch

```
int month= 6 ;
switch (month)
{
 case 1:  printf("JAN");break;
 case 2:  printf("FEB");break;
   . . .
 case 12: printf("DEC");break;
 default: printf("error");
 break;
}
```

If-else

```
int num = 6 ;

if ( num < 0)
{
   printf("negative");
}
else
{
   printf("positive");
}
```

- Used for many branches instead Of nested if-else
- Abscense of **break** will prevent the instruction to be printed and it will print the statement that has the first break.
- In if-else statement the else is related to the closest if
- More than one statement inside the if part or else part will need curly braces for the instructions

Dr. Amr Talaat

Operators

ASSUME we have to operands a and b

Logical negation (NOT)	!a
Logical AND	a && b
Logical OR	a || b

Logical operators

Equal to	a == b
Not equal to	a != b
Greater than	a > b
Less than	a < b
Greater than or equal to	a >= b
Less than or equal to	a <= b

Comparison operators

Bitwise NOT	~a
Bitwise AND	a & b
Bitwise OR	a | b
Bitwise XOR	a ^ b
Bitwise left shift[note 1]	a << b
Bitwise right shift[note 1]	a >> b

Bitwise operators

Cont.

ASSUME we have to

Addition assignment	a += b
Subtraction assignment	a -= b
Multiplication assignment	a *= b
Division assignment	a /= b
Modulo assignment	a %= b
Bitwise AND assignment	a &= b
Bitwise OR assignment	a |= b
Bitwise XOR assignment	a ^= b
Bitwise left shift assignment	a <<= b
Bitwise right shift assignment	a >>= b

Compound operators

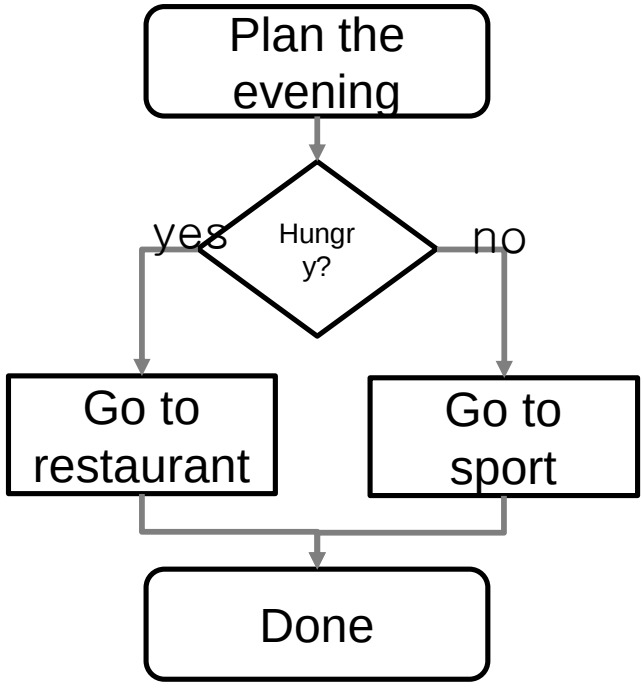
Basic assignment		a = b
Addition		a + b
Subtraction		a - b
Unary plus (integer promotion)		+a
Unary minus (additive inverse)		-a
Multiplication		a * b
Division		a / b
Modulo (remainder)		a % b
Increment	Prefix	++a
	Suffix	a++
Decrement	Prefix	--a
	Suffix	a--

Arithmetic operators

Dr. Amr Talaat

Dr. Amr Talaat

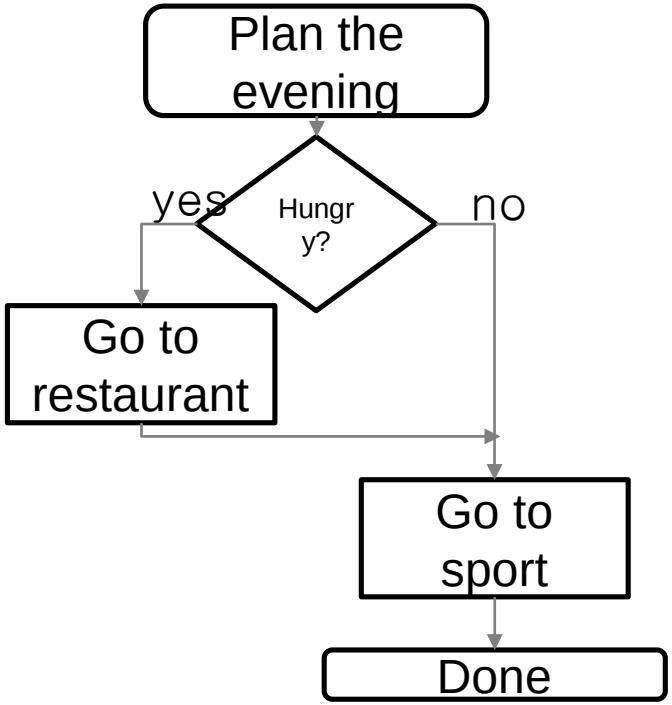
Basic Concepts: Branch




```
void PlanEvening(char hungry)
{

    if (hungry)
    {
        goRestaurant();
    }
    else
    {
        goSport();
    }

}
```

Dr. Amr Talaat

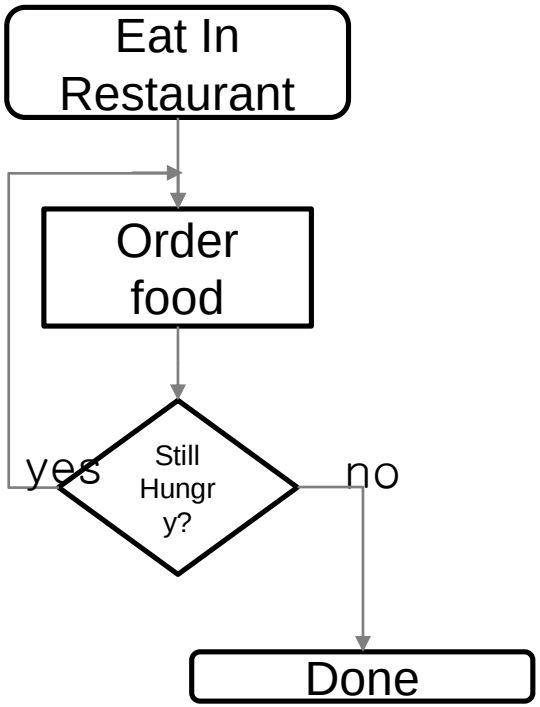
Basic Concepts: Branch




```
void PlanEvening(char hungry)
{

    if (hungry)
    {
        goRestaurant();
    }


    goSport();


}
```

Dr. Amr Talaat

Basic Concepts: Loop

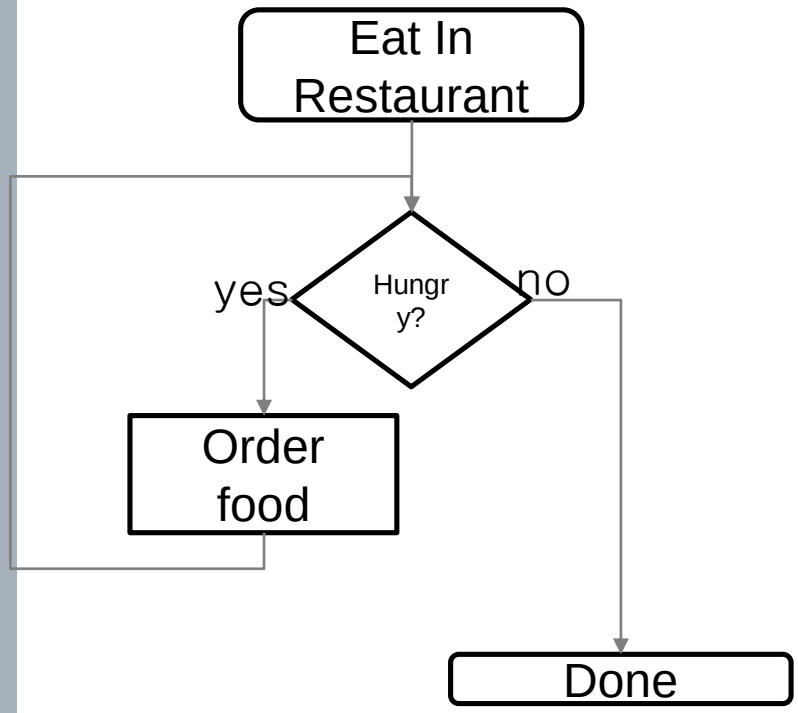



```
void EatInRestaurant()
{
   do
   {
      OrderFood();


   } while (CheckIfHungry());
}
```

Semicolon

Dr. Amr Talaat

Basic Concepts: Loop

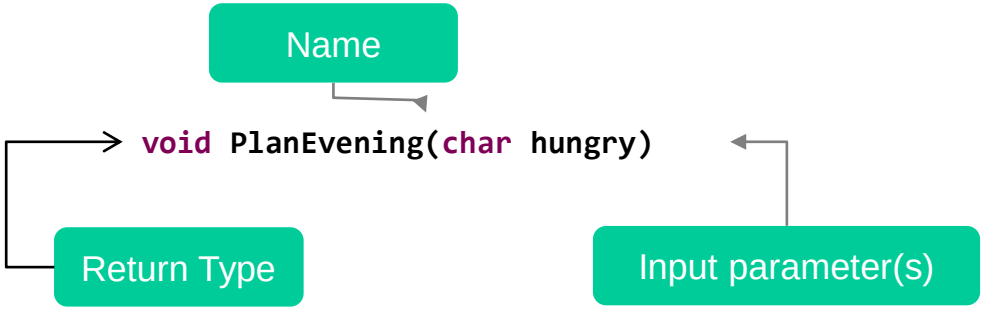



```
void EatInRestaurant()
{
  while (CheckIfHungry())
  {


    OrderFood();
  }


}
```

Semico

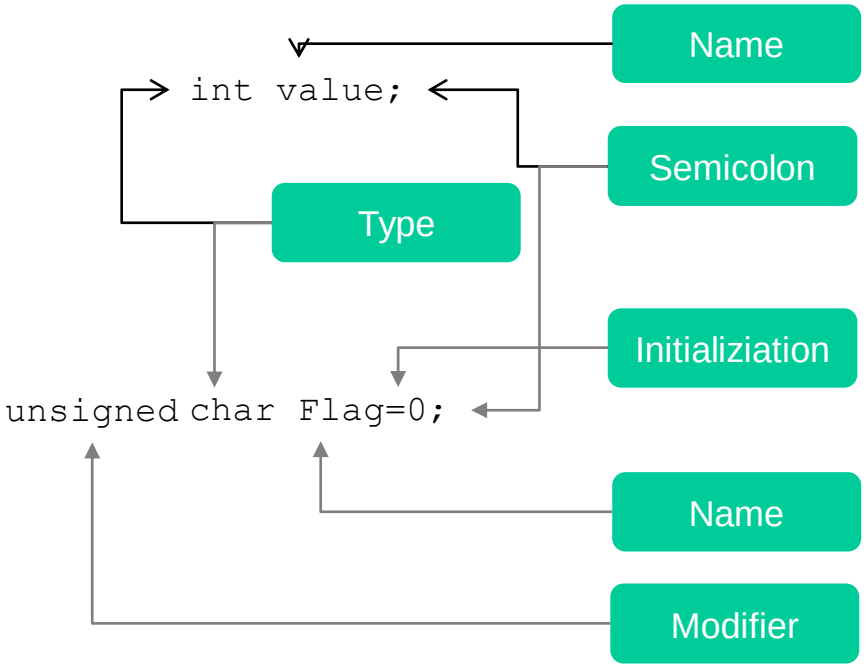
Dr. Amr Talaat

Basic Concepts: Function




```
#define Yes 1
#define No 0

PlanEvening(Yes);
```

Use of variables: Declare before Use



Dr. Amr Talaat

Basic Data Types

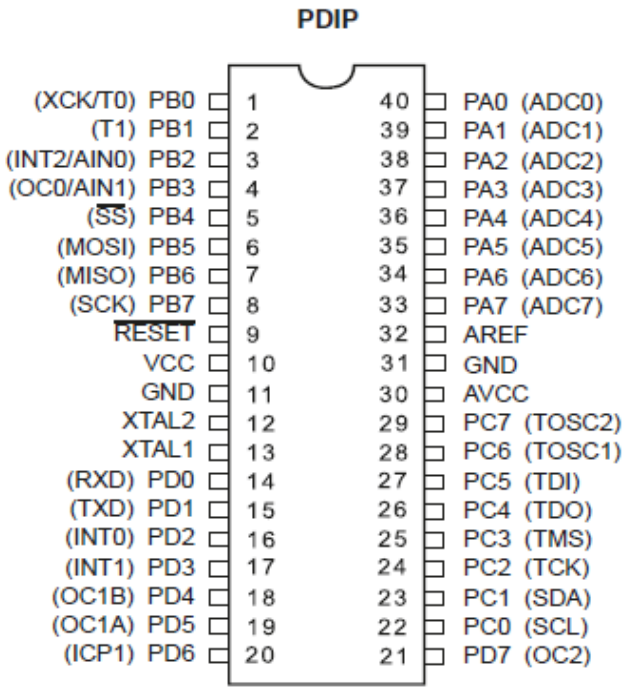
char	1 Byte (8 Bit)	-128 ... +127	0 ... 255
short	2 Byte (16 Bit)	-32768 ... +32767	0 ... 65535
int	2 Byte, 16 Bit	-32768 ... +32767	0 ... 65535
long	4Byte (32 Bit)	-2147483648 ... +2147483647	0 ... 4294967295

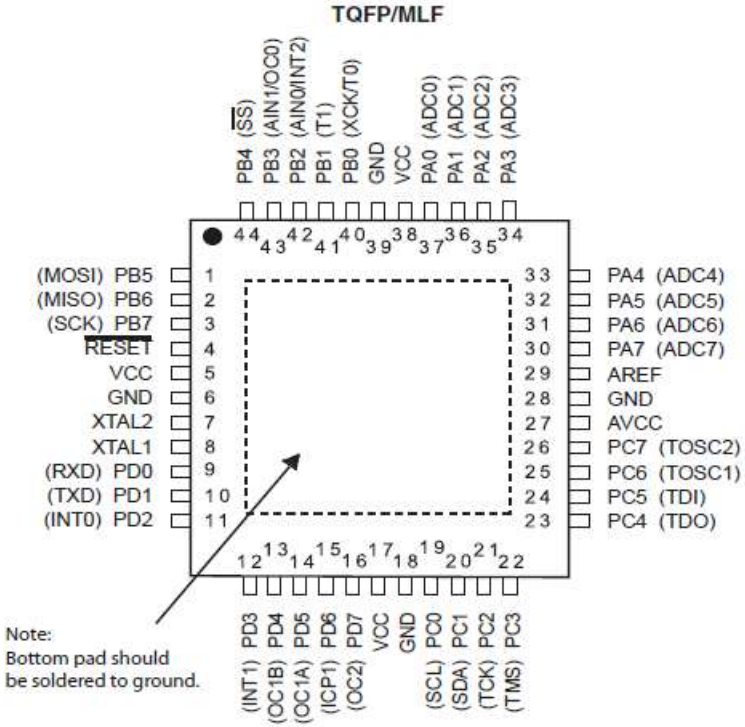
And bool?

float	4 Byte (32 Bit)	$-3{,}402823466 \cdot 10^{38}$ ... $-1{,}175494351 \cdot 10^{-38}$ 0 $+1{,}175494351 \cdot 10^{-38}$ ... $+3{,}402823466 \cdot 10^{38}$
double	8 Byte (64 Bit)	$-1{,}7976931348623157 \cdot 10^{308}$ ... $-2{,}2250738585072014 \cdot 10^{-308}$ 0 $+2{,}2250738585072014 \cdot 10^{-308}$ ... $+1{,}7976931348623157 \cdot 10^{308}$

Dr. Amr Talaat

Pin assignment



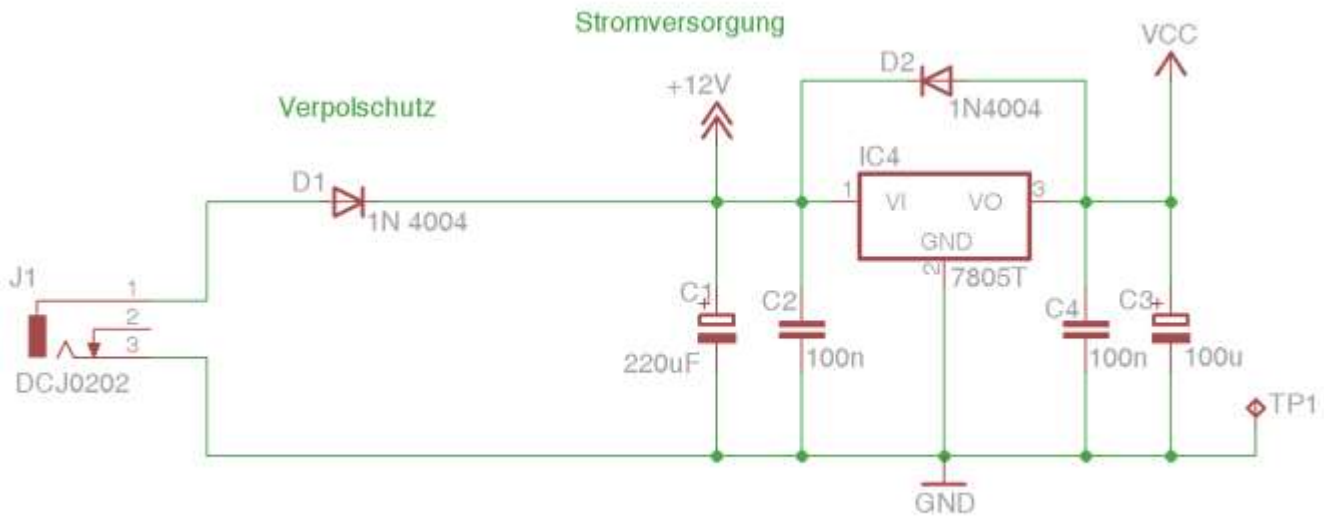


Source: Microchip

The Meaning of the Pins

- Oscillator/Voltage/Reset (XTAL, VCC, GND, Res et)
- Digital I/O („Ports“)
- Special purpose Digital I/O (e.g. OCxA/B)
- Analog Input („ADC“) and support (AREF, AVCC)
- Serial Communication
  - USART: Rx, Tx
  - SPI: MISO, MOSI, SS, SCK
  - TWI: SCL, SDA

This is a typical voltage regulator



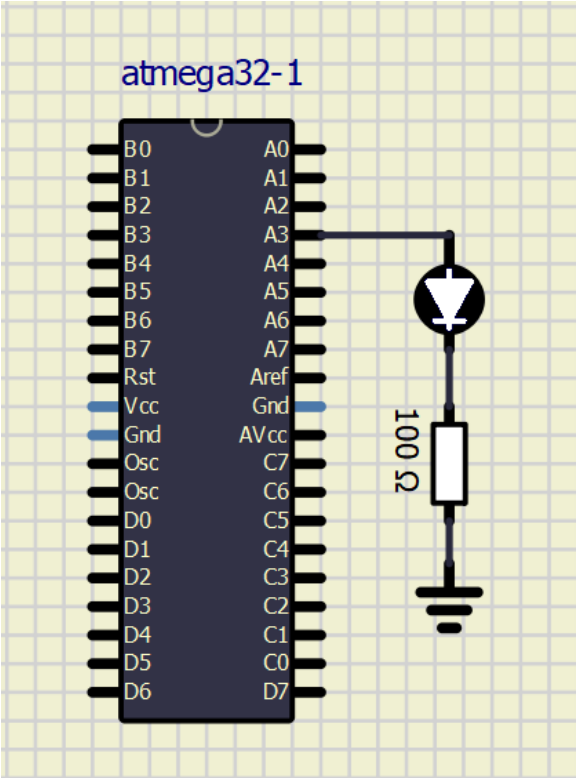
Dr. Amr Talaat

How to access to the Digital I/O

- Microchip AVR 8-bits microcontrollers provide pi ns to take in/output information from/to the out side world in the form of logic values. These pin s are usually organized in groups of eight (8) an d referred to as a port.
- The AVR use the alphabet to name these port, e xample PortA, PortB, etc.
- The pins of PortB are: PB0 - PB7.
- ATMega32 has got PortA (8 Pins), PortB (8 Pins) , PortC (8 Pins) and PortD (8 Pins)

http://www.avr-tutorials.com

Now, let‘s see how to turn on an LED



The Port is A3
so:

Set the Data Direction Register of
`PA0` to „Output“ (Write 1)

```
DDRA |= (1<<DDA3);
```

because in avr/iomx32.h which is included by avr/io.h

```
#define DDRA     _SFR_IO8 (0x1A)
/* DDRB */
#define DDA7     7
#define DDA6     6
…
#define DDA3     3
…
#define DDA0     0
…
```

and then:

```
PORTA |= (1<<PA3);
```

Access to a single bit: Setting

PORTA	x	x	x	x	x	x	x	x
				OR				
0x80 (Hex) or 1 << 3 or 1 << PA3	0	0	0	0	1	0	0	0
	x	x	x	x	1	x	x	x

PORTA |= (1<<PA3);

By Bool's law: $a \vee 1 = 1$; $a \vee 0 = a$

Dr. Amr Talaat

Combination of Several Bits

PORTA	x	x	x	x	x	x	x	x
				OR				
0x80 (Hex) or 1 << 3	0	0	0	0	1	0	0	0
				OR				
0x40 (Hex) or 1 << 5	0	0	1	0	0	0	0	0
	x	x	1	x	1	x	x	x

```
PORTA |= (1<<3) | (1 << 5);
```

Dr. Amr Talaat

Access to a single bit: CLEARING

PORTA	x	x	x	x	x	x	x	x
				AND				
0xF7 (Hex) or ~(1 << 3)	1	1	1	1	0	1	1	1
	x	x	x	x	0	x	x	x

PORTA &= ~(1<<3);

By Bool's law: $a \wedge 1 = a$; $a \wedge 0 = 0$

By bitwise inversion: (1<<3) = 0b00001000
~(1<<3) = 0b11110111

Dr. Amr Talaat

Toggling a Bit

PORTA	x	x	x	x	1	x	x	x
					XOR			
0x40 (Hex) or 1 << 3	0	0	0	0	1	0	0	0
PORTA ^= (1<<3);	x	x	x	x	0	x	x	x

By Bool's law: a XOR 1 = ¬a; a XOR 0 = a

Dr. Amr Talaat

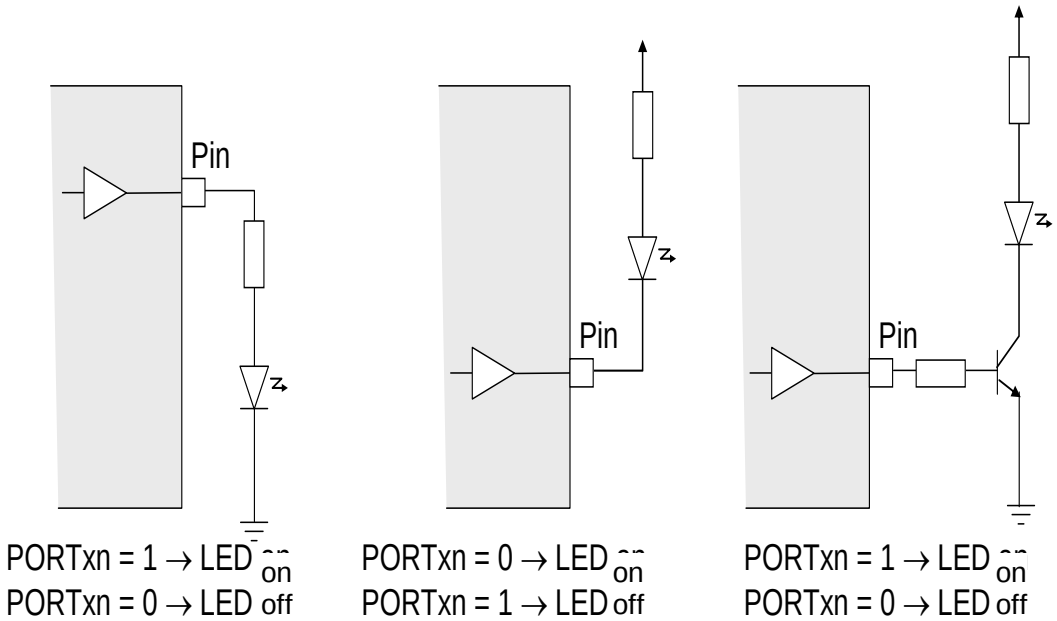
Toggling a Bit

PORTA	x	x	x	x	0	x	x	x
					XOR			
0x40 (Hex) or 1 << 3	0	0	0	0	1	0	0	0
PORTA ^= (1<<3);	x	x	x	x	1	x	x	x

By Bool's law: a XOR 1 = ¬a; a XOR 0 = a

Dr. Amr Talaat

Some possible variants



`(1<<DDD6)` corresponds to `0100 0000` `(left shift of 1 by 6)`

`DDRD |= (1<<DDD6)` means `DDRD` then is `x1xx xxxx`

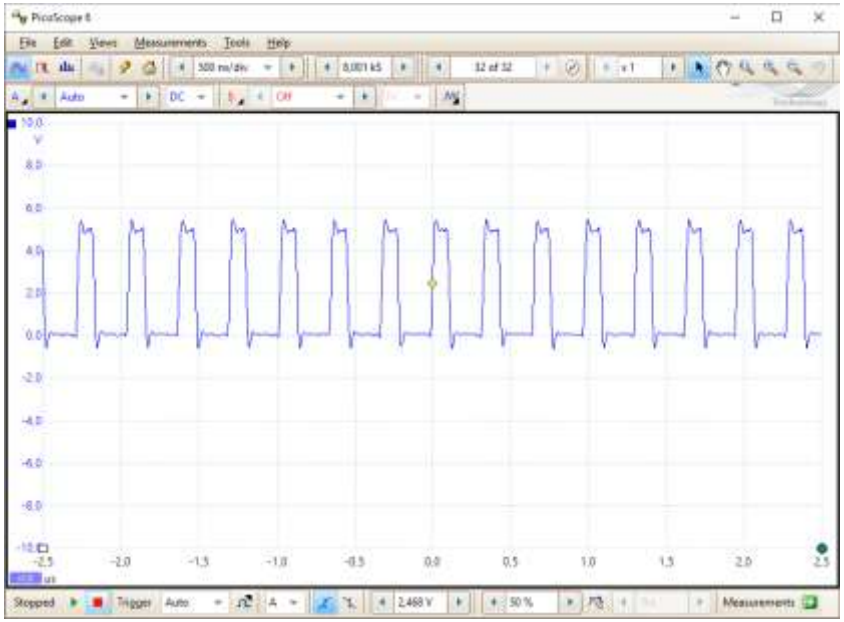
Where x means the value before the operation. „|“ is the bitwise OR operator

`DDRD &= ~(1<<DDD6)` means `DDRD` then is `x0xx xxx`

`&` is the bitwise AND operator

Dr. Amr Talaat

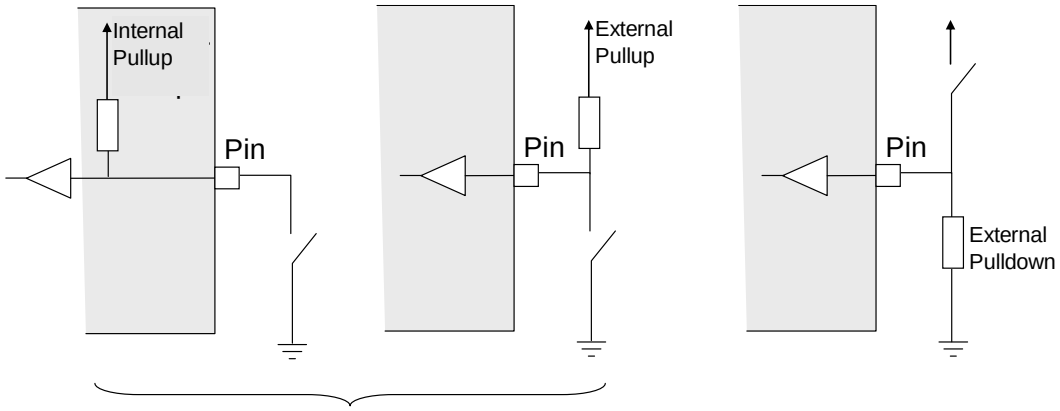
See, how it looks like when we switch the LED on and OFF




```
int main(void)
{/* Initialization part */
DDRD |= (1<<DDD6); #//set Data
direction register to output
while(1)
    {
            PORTD |= (1<<PD6);
            PORTD &=
~(1<<PD6);

    }
}
```

Dr. Amr Talaat

Now, see how we can read from Input

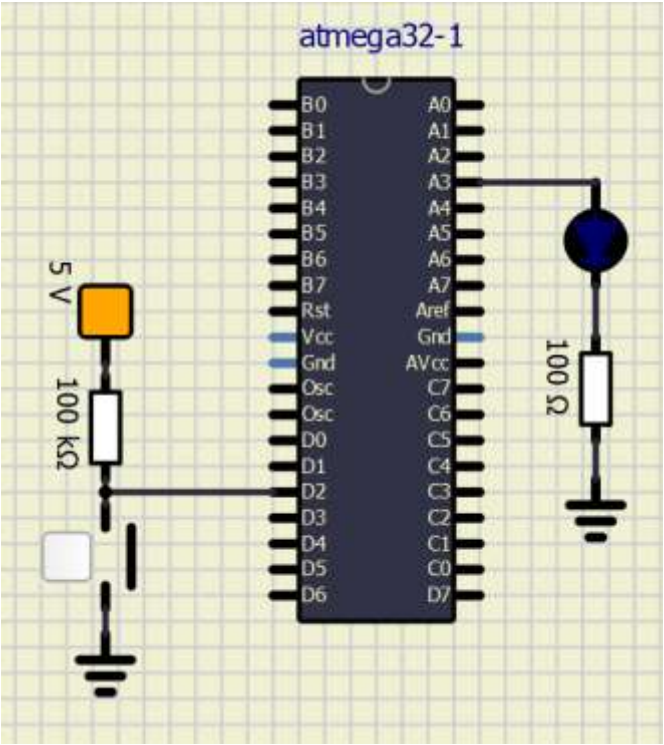


Key pressed: PIN goes to „0“
Key released: PIN goes to „1“

Key pressed: PIN goes to „1“
Key released: PIN goes to „0“

Dr. Amr Talaat

We use variant #2



- External Pullup
- First, set the DDR to „0"
- Second, read the stat e of the switch

Now, your turn!
Tell me the code for switching LED on/off by Switch

Let‘s study the whole code

x	x	x	x	?	x	x	x
0	0	0	0	1	0	0	0

We do it in a endless loop

Once the condition is valid, the LED stays on

So we have to switch it off again

```
int main()
{
        Init(); // Initialisation

        while(1)  //endless loop
        {                                   // Here, we will place the
tasks

                if (!(PIND & (1<<PD2)))             // switch is

                {
                        PORTA |= (1<<PA3) ; //LED is On
                }
                else
                {
                        PORTA &= ~(1<<PA3);     // LED is Off
                }
        }
        return 0;
}
```

Later on, we have to consider de-bouncing…

Summary Digital I/O

Operation mode	DDRx	PORTx	PINx	Electrical characteristics
Input (tristate)	0	0	Open	tristate
Input (high)	0	1	1 (0 when grounded)	High over ≈ 30 k Ω
Output	1	0	0	Low
Output	1	1	1	High

..at some µC of the AVR familiy, writing 1 into PINx in output mode toggles the PORT.
You can also toggle by e.g. `PORTD ^= (1<<PD6)` because the PORT register holds the value of the outpunt state and ^ means bitwise exclusive or

…now we should organize it a little

- We structurize the program in a main module an d in functional modules, each of them having a . c File (Source) and a .h File (Header)
- Right now, only the main module is sufficient
- Functions (Subroutines) will be used whenever it is appropriate, e.g.
  - If a function does one job repeatedly
  - If a function is very long and should be partiti oned
  - If a function is called from many other functio ns or is part of an interface
  - …
- See example

Dr. Amr Talaat

Assignment 1

Dr. Amr Talaat